

Curro Hillcrest Service tracker

Ruth-ann Rudolph

Grade 12

2022

Contents

User Interface Design 4
1. Log-In 4
2. Sign-up 5
3. Home 6
4. Edit account details 7
5. Events 8
6. Participation 9
7. Home Admin 10
8. Edit Admin account 11
9. New Admin account 12
10. Events Admin 13
11. Create new event 14
12. Edit Event 15
13. Participation Admin 16
14. Students 18
15. Help 19
Sequencing and algorithms 20
Log-In screen: 20
1. Log in button pressed: 20
2. Sign up button pressed: 21
3. Help button pressed: 21
Sign-up screen: 21
1. Sign-up button pressed: 21
2. Help button pressed 23
Home 24
1. Edit Account Details button pressed 24
2. Help button pressed 25
Edit Account 25
1. Enter button pressed: 26
2. Cancel button pressed: 27
Events: 28
1. Search by date button pressed 28

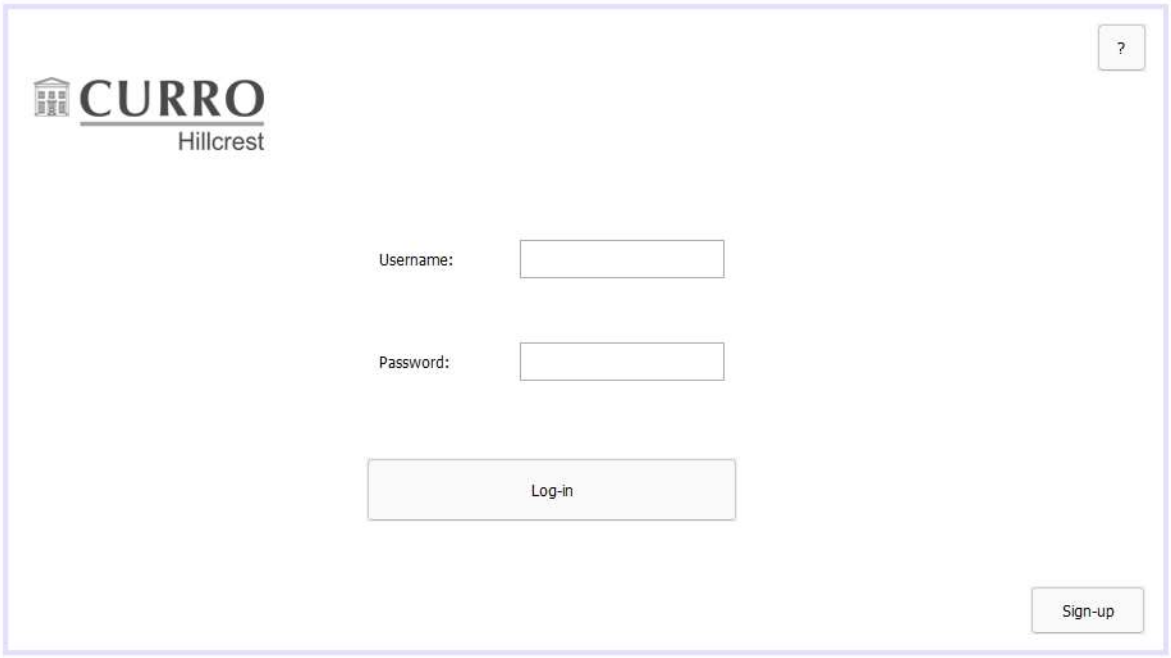
2. Search by name button pressed 29
3. Search by ID button pressed 30
Participation: 32
1. Search by portfolio button pressed 32
2. Search by event name button pressed 34
3. Search by event ID button pressed 35
4. New participation button pressed 36
5. Delete participation button pressed 36
Admin home page: 39
1. Edit account button pressed: 39
2. Create new admin account button pressed: 40
3. Help button pressed: 40
Edit admin account page: 41
1. Enter button pressed: 41
2. Cancel button pressed: 42
New Admin Account: 42
1. Enter button pressed: 43
2. Cancel button pressed: 44
Admin Events page: 44
1. Search by date button pressed: 44
2. Search by name button pressed: 46
3. Search by ID button pressed: 47
4. Create new event button pressed: 48
5. Edit event button pressed: 49
6. Help button pressed: 50
Create new event: 50
1. Add button pressed 50
2. Cancel button pressed: 51
Edit event: 52
1. Find event button pressed: 52
2. Enter button pressed: 54
3. Cancel button pressed: 55
Admin participation page: 56

1. Search by portfolio button pressed: 56
2. Search by event name button pressed: 57
3. Search by student name button pressed: 59
4. Search by event ID button pressed: 61
5. Create record button pressed: 62
6. Delete record button pressed: 62
7. Help button pressed: 64
Students page: 64
1. Search by student ID button pressed: 65
2. Search by student name button pressed: 67
3. Search by grade button pressed: 70
4. Help button pressed: 72
Help page: 73
X button pressed: 73
Help button pressed: 74
Class diagrams 74
Objects 74
Managers 76
Persistent storage design 78
1. tblStudent 78
2. tblAdmin 79
3. tblEvent 80
4. tblParticipation 80
Explanation of Storage Design **Error! Bookmark not defined.**
Primary memory 81
Student Class 81
Admin Class 81
Event Class 81
Participation Class 81
DB Class 82
StudentManager class 82
AdminManager class 82
EventManager class 82

ParticipationManager 82
Secondary Memory 82
Students table 82
Admins table 83
Events table 83
Participation table 83
UsernamesAndPasswords.txt 83

User Interface Design

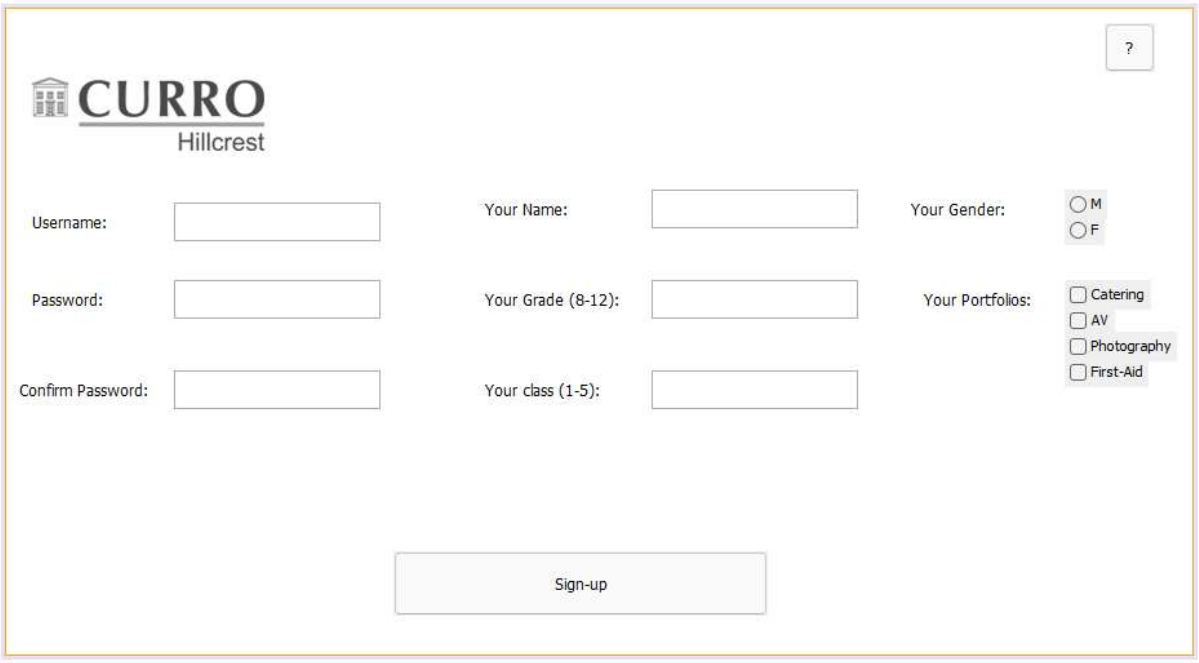
1. Log-In



Description	This screen will allow a user to sign into an existing account by entering a valid username and password or navigate to the Sign-up page to create a new Student account
Data	• The user's username and password once entered in the text field • Error messages will appear in a pane
Actions	Login button • The user's username and password will be compared to the database and if correct, the Login screen will close and the Home screen will be shown.

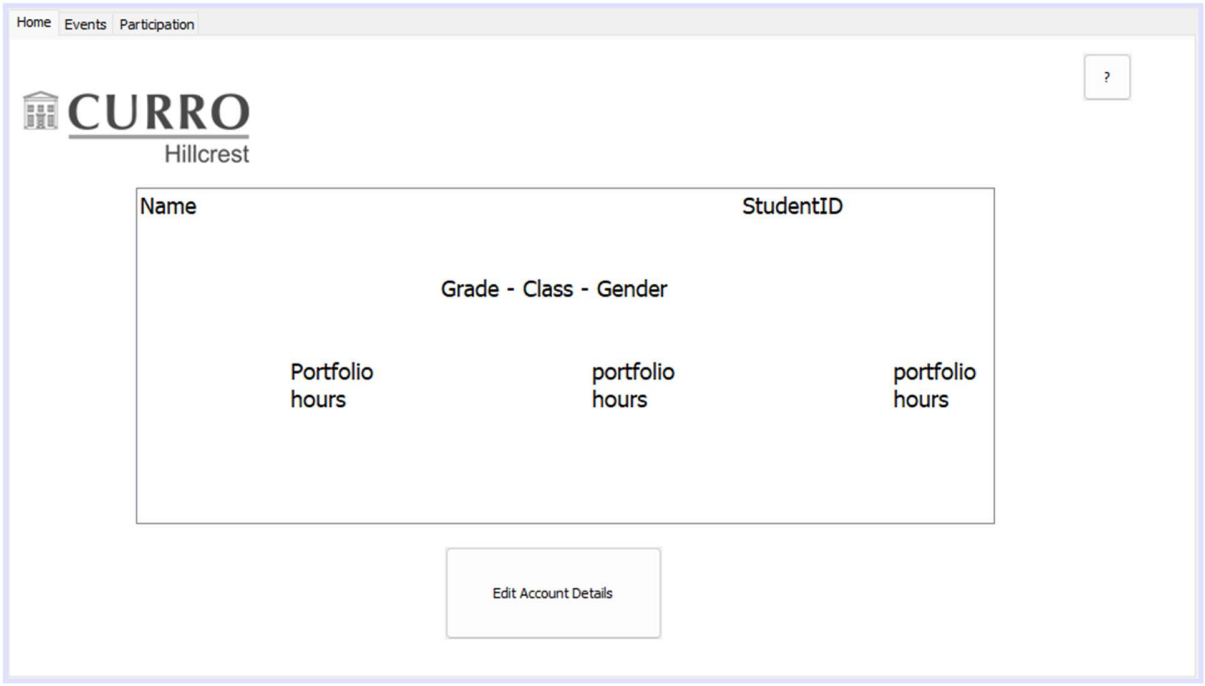
	• If incorrect, an error message will be displayed. Sign Up button • The user will be directed to the Sign-up page The Help button • Will take the user to the help screen

2. Sign-up



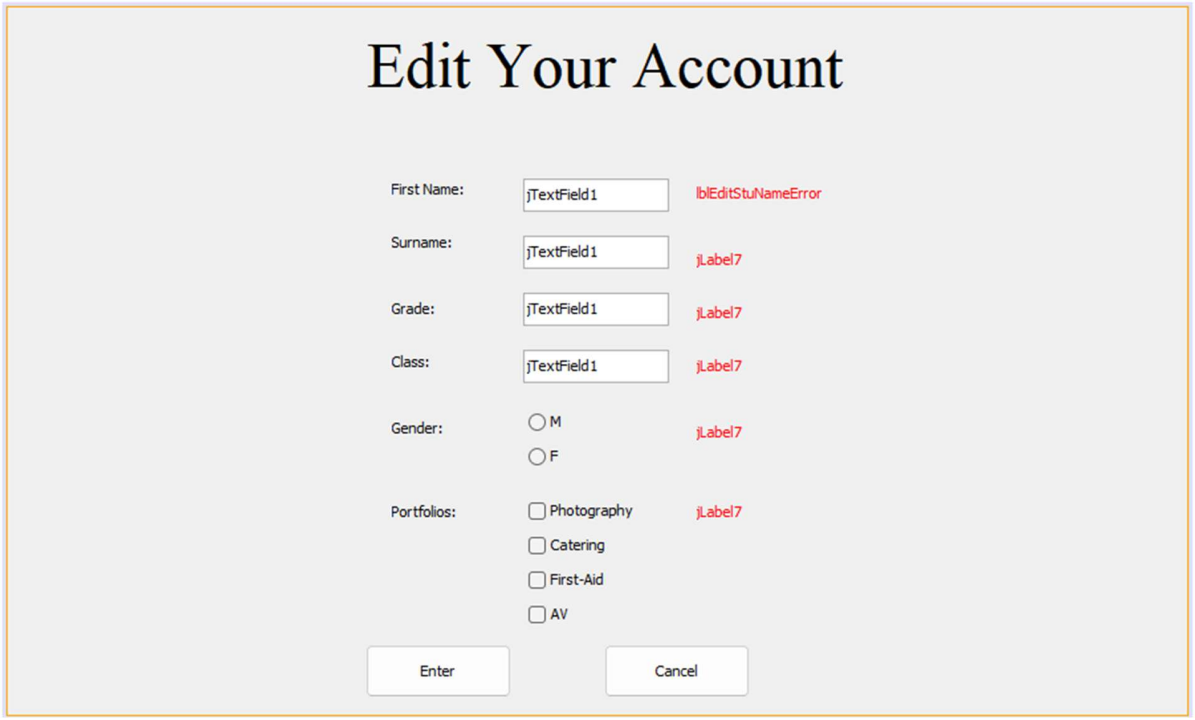
Description	This screen allows new users to enter all their information into relevant text fields so that they can create a new account.
Data	• All of the users information entered into the text fields and selected buttons (Username, password, name, grade, class, gender, portfolios) • Error messages will appear in a pane
Actions	Sign-up button • When selected the program will check that all information has been entered before a new account is created. • If there is information missing a pane will appear to tell the user that all fields must be filled out. The Help button • Will take the user to the help screen

3. Home



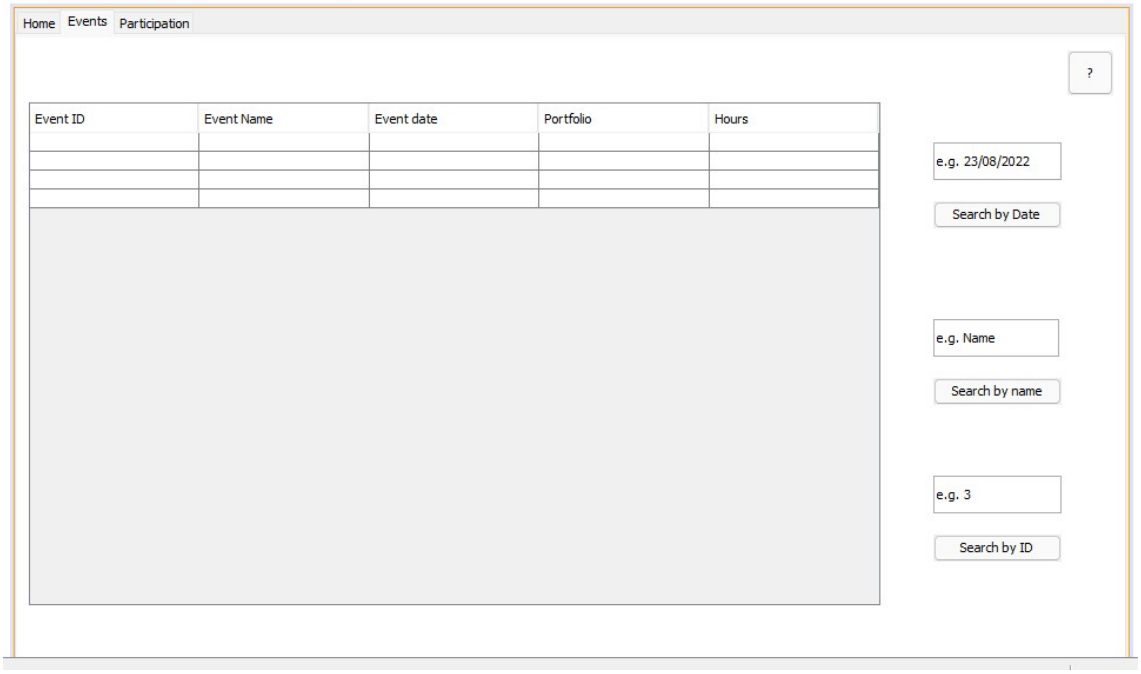
Description	This screen allows a student to view all of their information and to press a button which will direct them to a page where they can edit their account. They can also access other parts of the program or the help page.
Data	• All of the users information is displayed (Name, student ID, Grade, class, gender, portfolios and hours)
Actions	Edit account button • Will direct the user to a page where they can edit their account details The Help button • Will take the user to the help screen

4. Edit account details



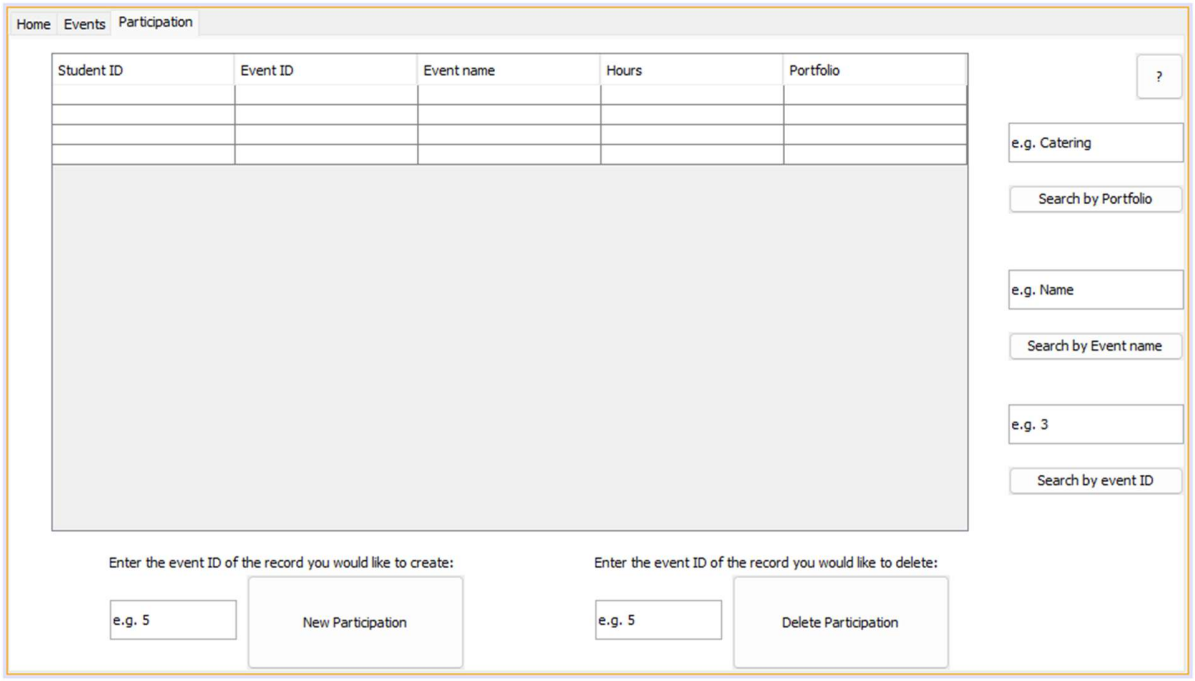
Description	This screen allows a student to view all of their information and change it , they are able to press a button which will direct close the page and cancel the change or they can enforce the changes they have made by pressing the enter button.
Data	• All the user's information will be displayed in the corresponding text fields, radio buttons or check boxes. • Any errors with the data entered will be displayed in the error labels
Actions	Enter button: • Will update the student's information in the database and the array, the edit account page will close, returning the user to the home page Cancel button: • Will discard any changes the user has made and close the edit page, returning the user to the home page

5. Events



Description	This screen will allow users to view all the events listed for the portfolios they participate in, search for events or navigate to other parts of the program.
Data	• All the details related to an event will be displayed in the table (ID, name, date, portfolio and hours) • The data entered into the text fields will be used to search for records
Actions	The search by date button • Allows users to enter a date, the database will be checked for events listed for that date and if any are found they will be displayed The search by Name button • Allows users to enter an event name, the database will be checked for events listed by that name and if any are found they will be displayed The search by ID button • Allows users to enter an event ID, the database will be checked for events listed with that ID and if any are found they will be displayed The Help button • Will take the user to the help screen

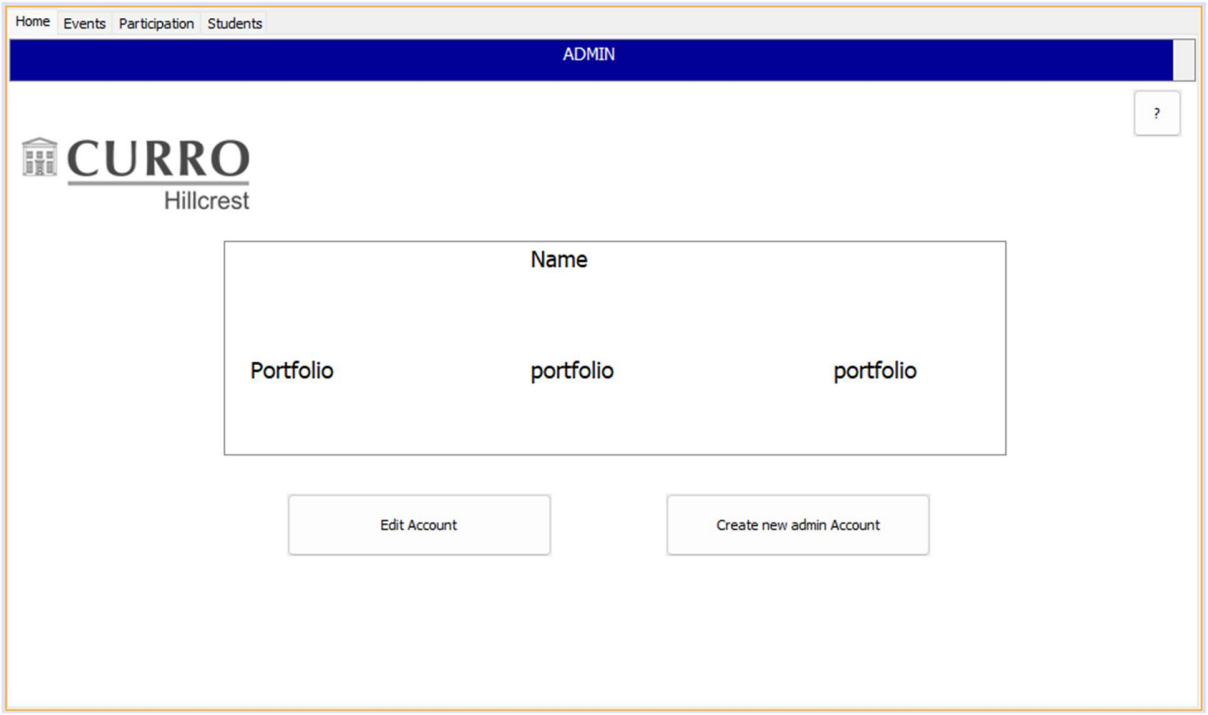
6. Participation



Description	This screen will allow users to view all the participation records listed for their name and search or participation records or navigate to other parts of the program.
Data	• All the details related to a participation record will be displayed in the table (ID, event ID, event name, portfolio and hours) • The data entered into the text fields will be used to search for records
Actions	The search by portfolio button • Allows users to enter a portfolio, the database will be checked for participation records listed for that portfolio and if any are found they will be displayed The search by Name button • Allows users to enter an event name, the database will be checked for participation records listed by that name and if any are found they will be displayed The search by ID button • Allows users to enter an event ID, the database will be checked for participation records listed with that ID

	and if any are found they will be displayed The new participation button: • Will take the information entered in the text field to the left and the student ID of the user logged in and add a new participation record to the database and the array. The delete participation button: • Will take the information entered in the text field to the left and the student ID of the user logged in and delete the participation record from the database and the array. The Help button • Will take the user to the help screen

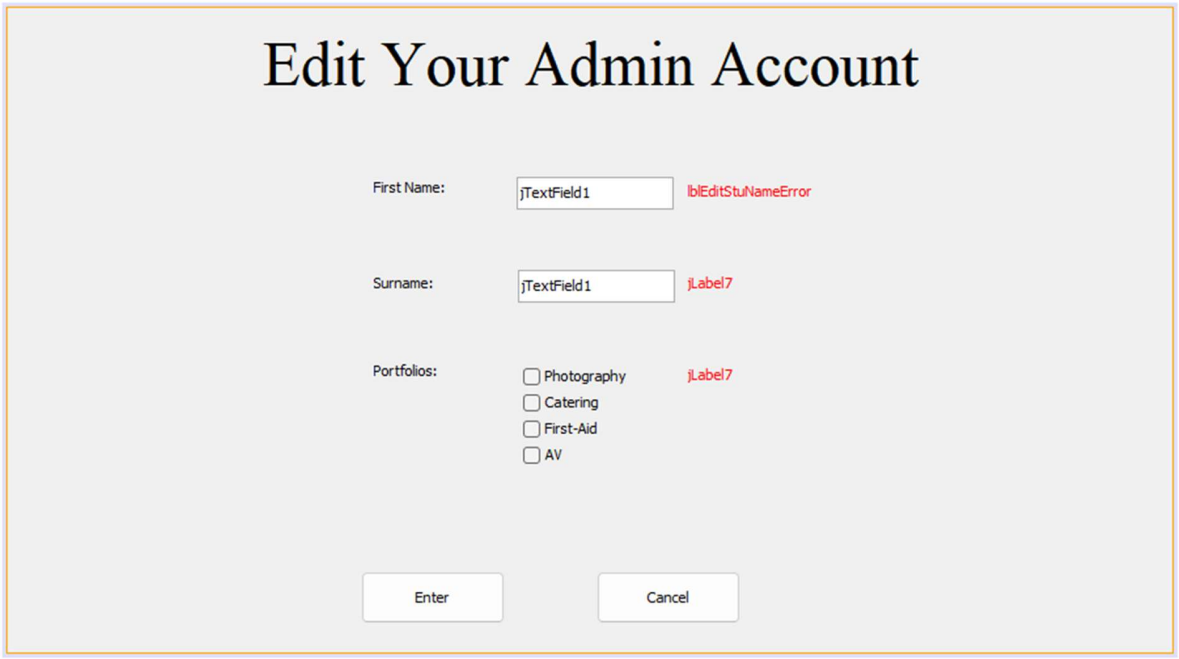
7. Home Admin



Description	This screen allows an admin user to view all their information and select a button to edit their account details. They can also access other parts of the program or the help page. Admins may also select a button to create new admin accounts from this page.

Data	• All their information is displayed (Name, portfolios) • The Edit account button • The new admin account button
Actions	Edit account button • Will redirect the user to a page where they can change their details Create new admin account button • Will redirect the user to a page where they can create a new admin account The Help button • Will take the user to the help screen

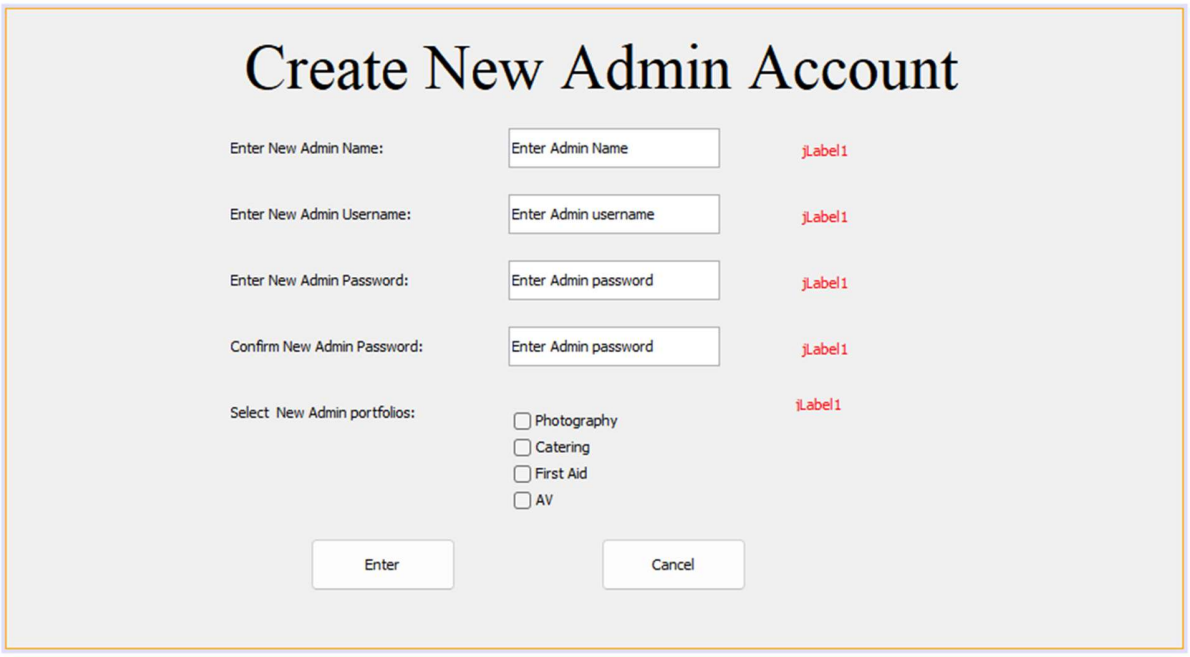
8. Edit Admin account



Description	This screen allows an admin to view all of their information and change it , they are able to press a button which will close the page and cancel the change or they can enforce the changes they have made by pressing the enter button.
Data	• All the user's information will be displayed in the corresponding text fields, radio buttons or check boxes. • Any errors with the data entered will be displayed in the error labels
Actions	Enter button: • Will update the admin's information in the database and the array, the edit

	account page will close, returning the user to the home page Cancel button: • Will discard any changes the user has made and close the edit page, returning the user to the home page

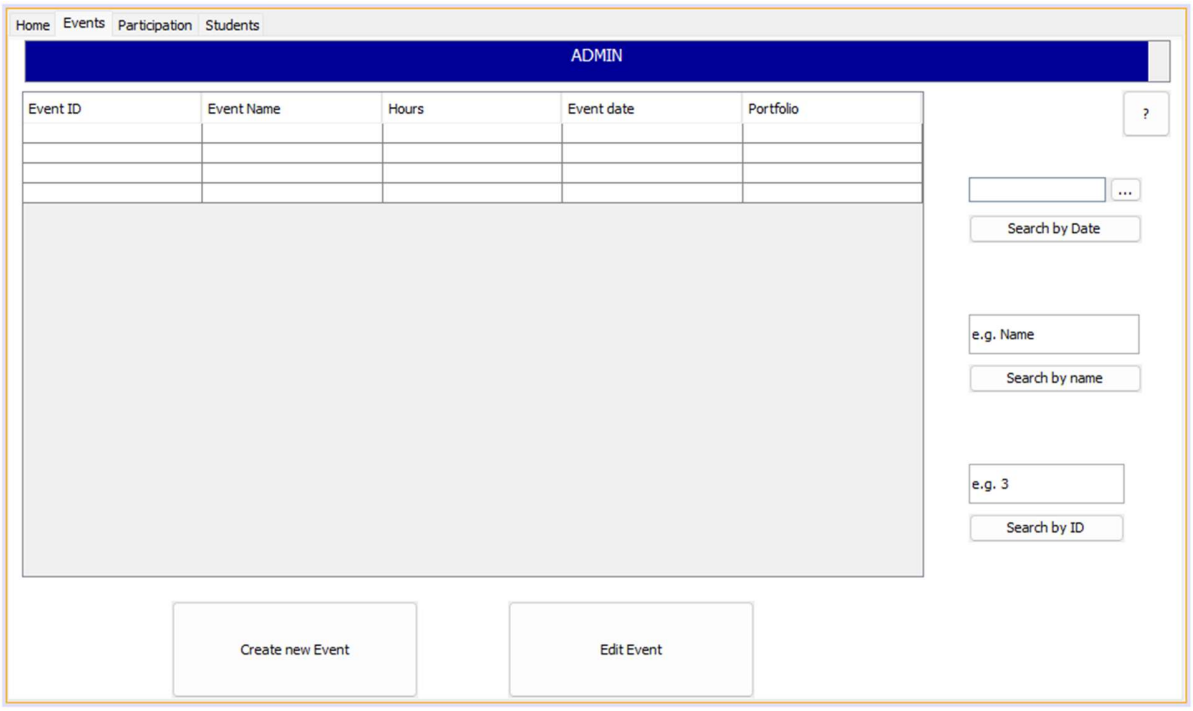
9. New Admin account



Description	This screen allows new users to enter all the information for the new admin into relevant text fields so that they can create a new admin account.
Data	• All of the users information entered into the text fields and selected buttons (Username, password, name, portfolios) • Error messages will appear in error labels
Actions	Enter button • When selected the program will check that all information has been entered before a new account is created. • If there is information missing or invalid an error will appear next to the invalid or missing data The cancel button

	• Will take the admin back to the home page and discard any information that has been entered

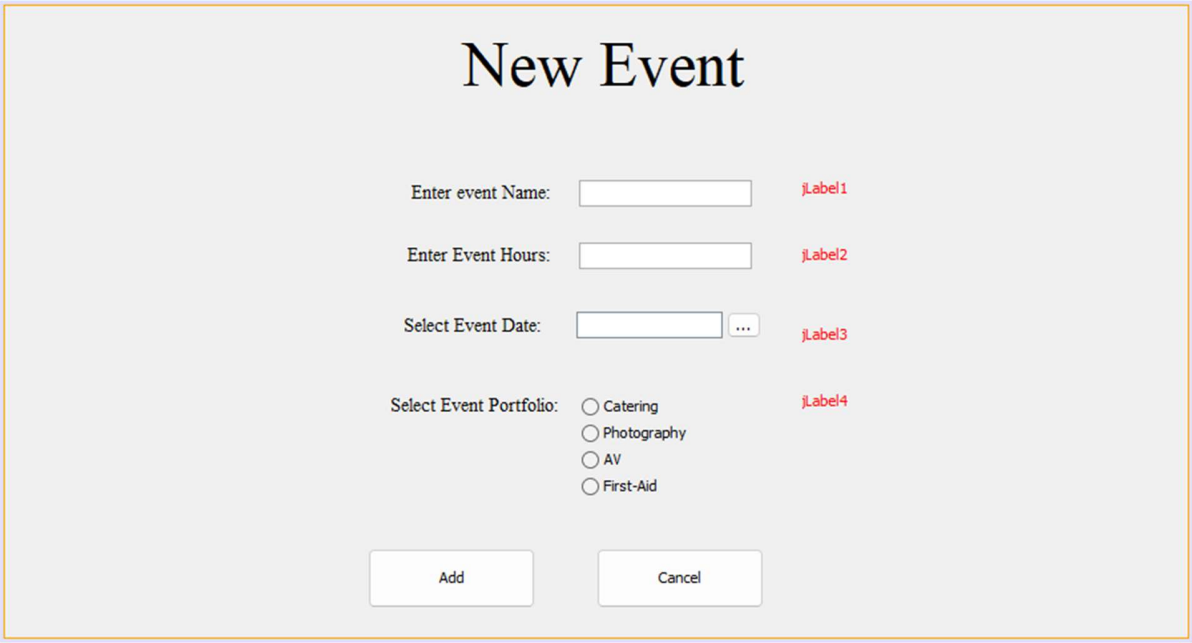
10. Events Admin



Description	This screen will allow users to view all the events listed for the portfolios they oversee, search for events, select a button to create a new event, select a button to edit an existing event or navigate to other parts of the program.
Data	• All the details related to an event will be displayed in the table (ID, name, date, portfolio and hours) • The data entered into the text fields will be used to search for records
Actions	The search by date button • Allows users to enter a date, the database will be checked for events listed for that date and if any are found they will be displayed The search by Name button • Allows users to enter an event name, the database will be checked for events listed by that name and if any are found they will be displayed The search by ID button

	• Allows users to enter an event ID, the database will be checked for events listed with that ID and if any are found they will be displayed The create new event button • Will redirect the user to a page where they may create a new event The edit event button • Will redirect the user to a page where they may edit an existing event The Help button • Will take the user to the help screen

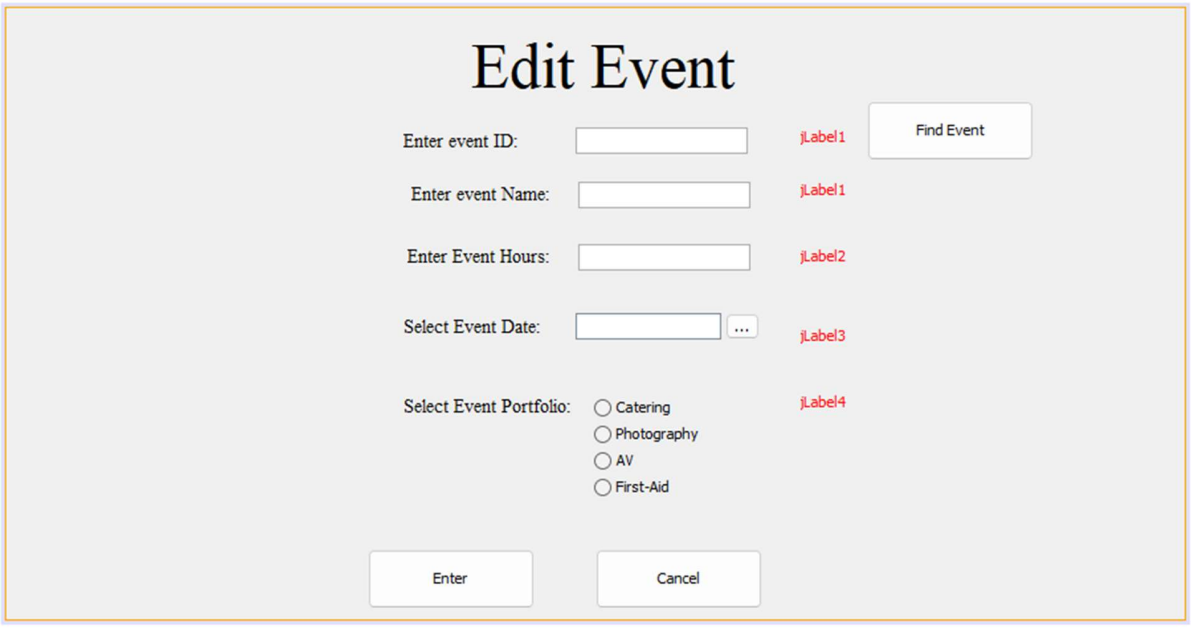
11. Create new event



Description	This screen allows new users to enter all the information for the new event into relevant text fields so that they can create a new event.
Data	• All of the event information entered into the text fields and selected buttons (name, hours, date, portfolios) • Error messages will appear in a error labels
Actions	Enter button • When selected the program will check that all information has been entered before a new event is created.

	• If there is information missing or invalid an error will appear next to the invalid or missing data The cancel button • Will take the admin back to the home page and discard any information that has been entered

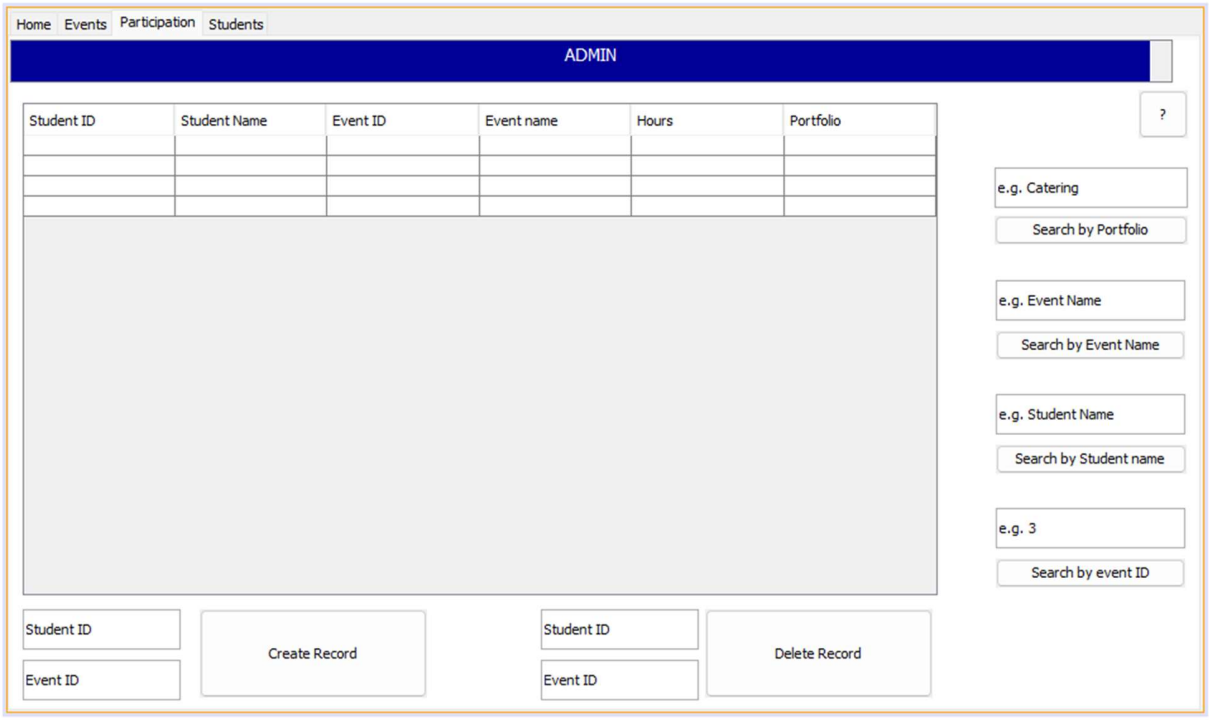
12. Edit Event



Description	This screen allows new users to edit all the information for the event in relevant text fields so that they can edit the event details.
Data	• Initially the user must enter the event ID into the Id field • All of the event information for the corresponding ID in the text fields and selected buttons (name, hours, date, portfolios) • Error messages will appear in a error labels
Actions	Find event button • The user must first enter the event ID, then the find event button will either give an error if the event ID does not match any events or the rest of the information will appear with the add button and text fields Enter button

	• When selected the program will check that all information has been entered before a new event is created. • If there is information missing or invalid an error will appear next to the invalid or missing data The cancel button • Will take the admin back to the home page and discard any information that has been entered

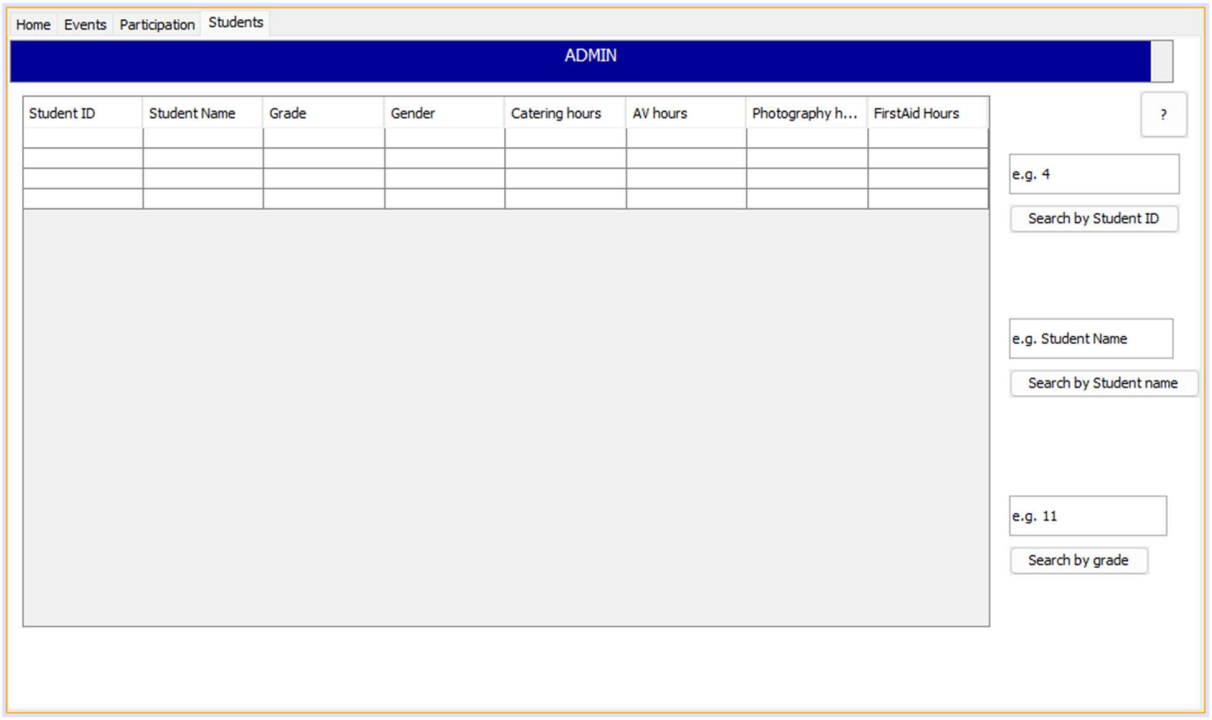
13. Participation Admin



Description	This screen will allow users to view all the participation records listed for their portfolios and search for participation records or navigate to other parts of the program.
Data	• All the details related to a participation record will be displayed in the table (ID, event ID, event name, portfolio and hours) • The data entered into the text fields will be used to search for records
Actions	The search by portfolio button • Allows users to enter a portfolio, the database will be checked for participation records listed for that

	portfolio and if any are found they will be displayed The search by event Name button • Allows users to enter an event name, the database will be checked for participation records listed by that name and if any are found they will be displayed The search by student Name button • Allows users to enter a student name, the database will be checked for participation records listed by that name and if any are found they will be displayed The search by ID button • Allows users to enter an event ID, the database will be checked for participation records listed with that ID and if any are found they will be displayed The delete button • Will check for a record listed with the Event ID and Student ID entered into the text fields and delete it from the database The create button • Will check the Event ID and Student ID entered into the text fields and create the record in the database The Help button • Will take the user to the help screen

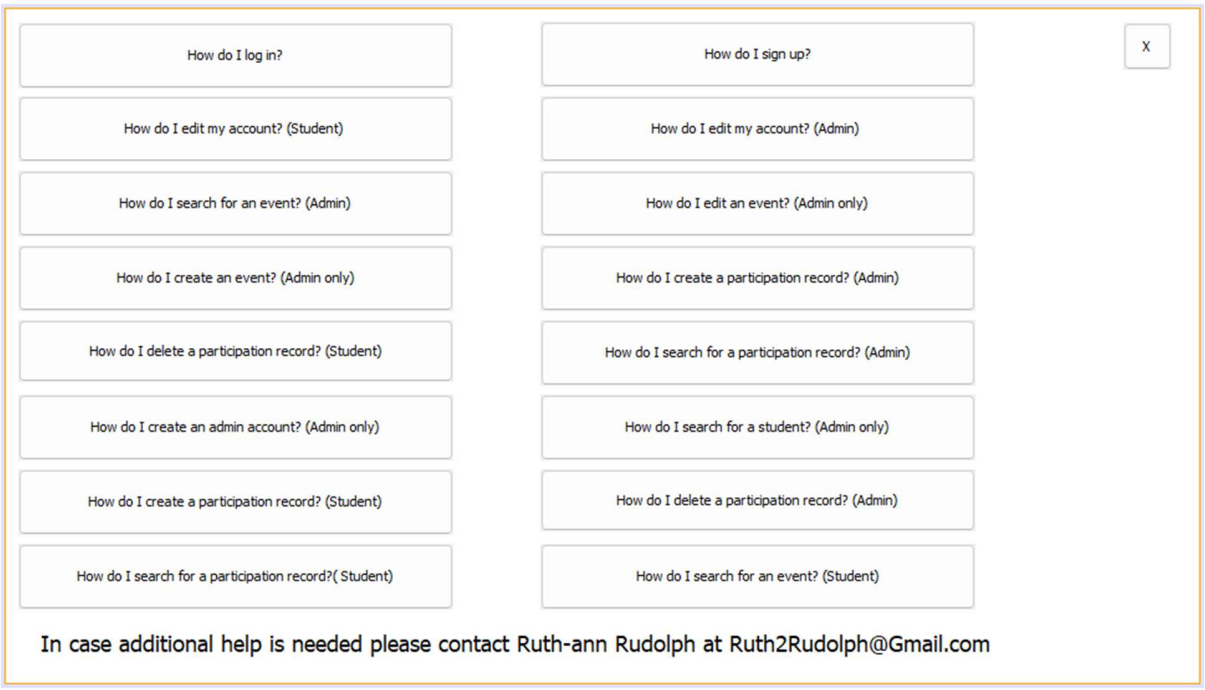
14. Students



Description	This screen will allow users to view all the Students listed for the portfolios they participate in, search for students or navigate to other parts of the program.
Data	• All the details related to an event will be displayed in the table (Student ID, student name, student surname, grade, class, gender, catering hours, photography hours, AV hours, First Aid hours) • The data entered into the text fields will be used to search for records
Actions	The search by grade button • Allows users to enter a grade, the database will be checked for Students listed for that grade and if any are found they will be displayed The search by Name button • Allows users to enter a student name, the database will be checked for students listed by that name and if any are found they will be displayed The search by ID button • Allows users to enter a student ID, the database will be checked for

	students listed with that ID and if any are found they will be displayed The Help button • Will take the user to the help screen

15. Help



Description	Will provide users with a list of possible questions they may have about how to perform certain tasks.
Data	No data
Actions	Each button will direct the user to a video of how to solve that problem or perform that task. The X button • Will return users to the page they were previously on.

Sequencing and algorithms

Log-In screen:



1. Log in button pressed:

```
WHILE (Scanner has next)
    line ← next line of scanner
    current ← ID from text file
    username ← username from text file
    password ← password from text file
    title ← title from text file
    IF (password = password from user AND username = username from user AND title =
    “Student”)
                    currentStu ← current
                    correct ← true
                    IF (currentStu IS NOT -1)
                            tbdpStudent set Visible
                     logIn screen disposed
                            pnlHome set Visible
                    END IF
            END IF
```

ELSE IF (password = password from user AND username = username from user AND title = “Admin”)

currentAdmin ← current

correct← true

tbdpAdmin set Visible

logIn screen disposed

END ELSE IF

END WHILE

IF (correct is false)

Display “Incorrect Username or password”

END IF

2. Sign up button pressed:

pnlSignUp set Visible

logIn screen disposed

calls method to clear all error labels

3. Help button pressed:

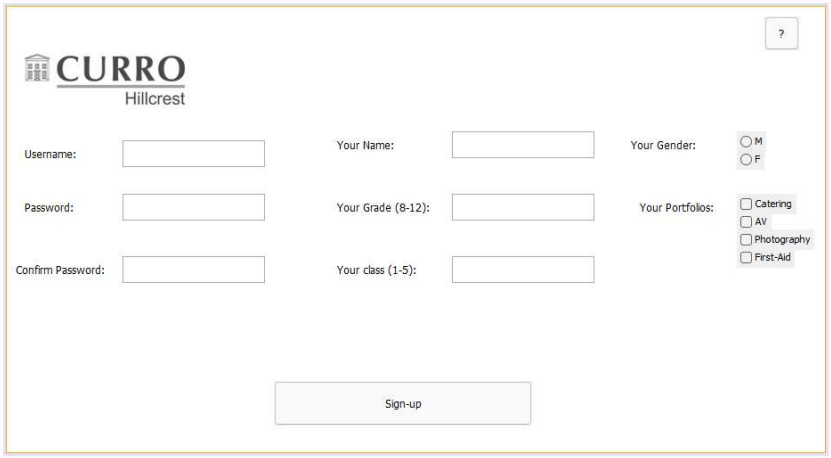
pnlHelp set Visible

pnlLogIn disposed

tbdpStudent disposed

help ← 1

Sign-up screen:



1. Sign-up button pressed:

Calls a method to clear all the error labels

```
IF (rbnNewStudentM is selected)
    gender ← 'M'
END IF
ELSE IF (rbnNewStudentF is selected)
    gender ←  'F'
END ELSE IF
ELSE
    lblNewStudentGenderError set text to "Please select a gender"
END ELSE
password ← false
IF (txfPassword2 = txfConfirmPassword1)
    password ← true
END IF
ELSE
    Display "Passwords Do Not Match"
END ELSE
validPortfolio ← calls method to check if there has been a valid portfolio selection made
validGrade ← calls a method to check if the type and range are valid
ValidClass ← calls a method to check if the type and range are valid
        validUserName ← calls a method to check if the presence and length are valid
        validPassword ← calls a method to check if the presence and length are valid
        validName ← calls a method to check if the presence, type and length are valid

IF (validUserName is true AND validPassword is true AND validName is true AND
gender IS NOT Blank AND validGrade is true AND ValidClass is true AND validPortfolio is true
AND password is true)
        name ← name entered by the user
        grade ← grade entered by the user
        classPoint ← class entered by the user
        lName ← splits the last name from the first name, from the name variable
        fName ← splits the first name from the last name, from the name variable
```

IF (the catering checkbox is selected)

catering ← true

END IF

IF (photography checkbox is selected)

photography ← true

END IF

IF (the first aid checkbox is selected)

firstAid ← true

END IF

IF (the AV checkbox is selected)

av ← true

END IF

Add Student (fName, lName , grade, classPoint, gender, catering, photography, av, firstAid);

line ← Last ID in the student array + "#" + username entered + "#" + password entered + "#Student”

currentStu ← Last ID in the student array plus 1

add line to the textfile

END IF

IF (new account has been created

pnlSignUp disposed

tbdpStudent set Visible

END IF

2. Help button pressed

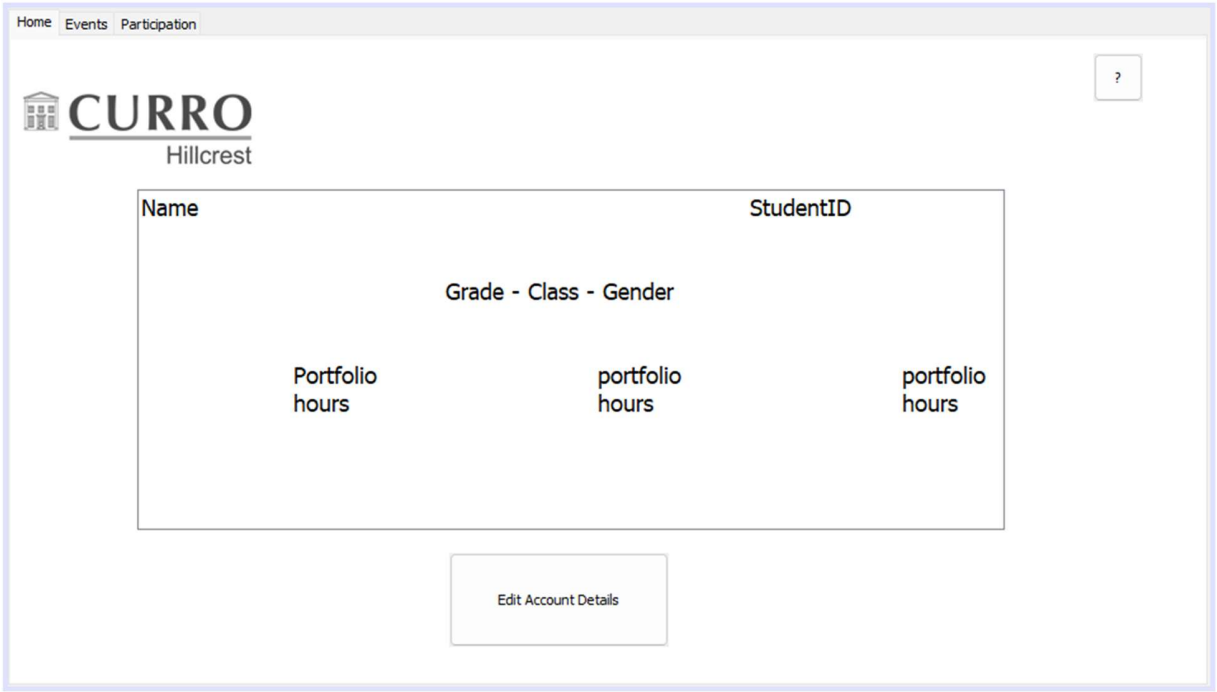
pnlHelp set Visible

pnlSignUp disposed

tbdpStudent disposed

help ← 2

Home



1. Edit Account Details button pressed

Calls a method to clear all the error labels

dlgEditStudent set visible

rs ← SELECT * FROM tblStudents WHERE StudentID = currentStu

while rs has next

IF (rs.getBoolean("Catering") is true)

cbxCatering1 set to selected

END IF

IF(rs.getBoolean("Photography") is true)

cbxPhotography1 set to selected

END IF

if(rs.getBoolean("FirstAid") is true)

cbxFirstAid1 set to selected

END IF

IF(rs.getBoolean("AV") is true)

cbxAV1 set to selected

END IF

txfEditStuName ← rs.getString("firstname")

txfEditStuSurname ← rs.getString("surname")

txfEditStuGrade ← rs.getString("Grade")

txfEditStuClass ← rs.getString("Class")

IF (rs.getString("Gender") = ‘F’)

rbnEditStuGenderF set to selected

END IF

ELSE IF (rs.getString("Gender") = ‘M’)

RbnEditStuGenderM set to selected

END ELSE IF

END WHILE

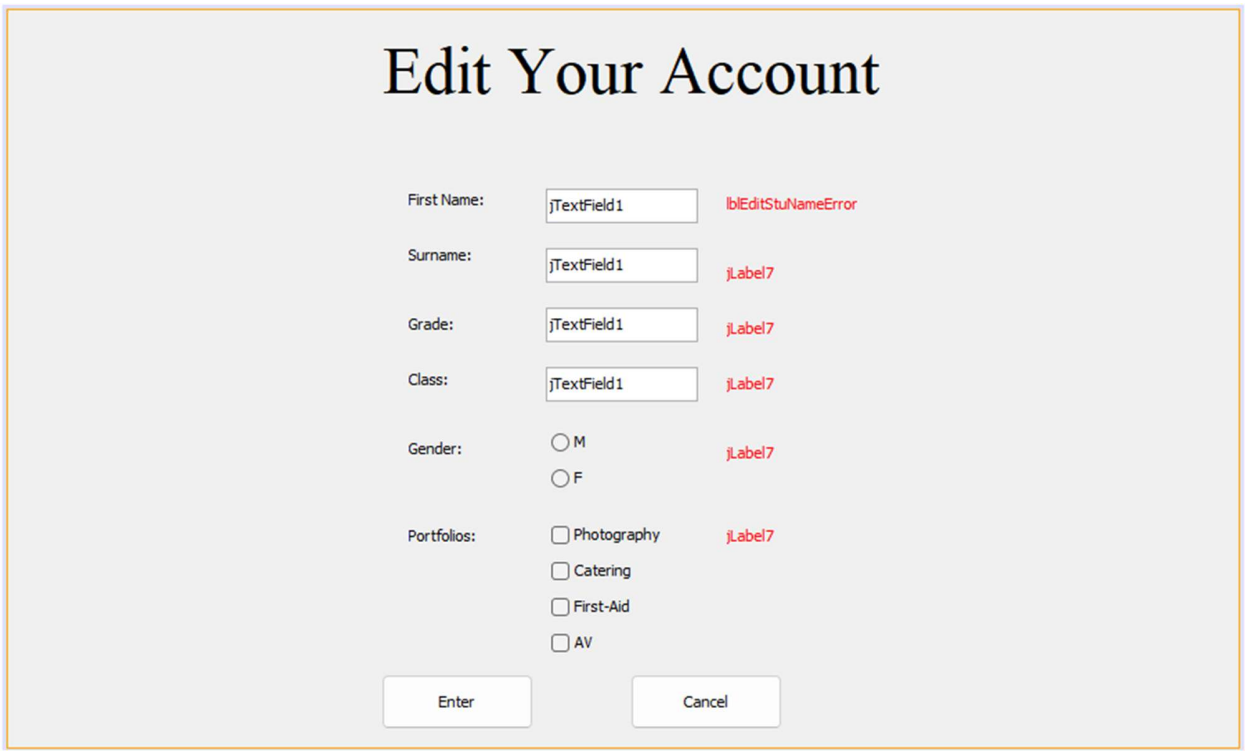
2. Help button pressed
   pnlHelp set Visible

   pnlSignUp disposed

   tbdpStudent disposed

   help ← 7

Edit Account



1. Enter button pressed:

Calls a method to clear all the error labels

gender ← ‘ ‘

IF (rbnEditStuGenderM is Selected)

gender ← 'M'

END IF

ELSE IF (rbnEditStuGenderF Is Selected)

gender ← 'F'

END ELSE IF

ELSE

lblEditStuGenderError ← "Please select a gender"

END ELSE

ValidPortfolio ← calls method to check if there has been a valid portfolio selection made

validGrade ← calls a method to check if the data is of the correct type, presence, and range

validClass ← calls a method to check if the data is of the correct Type, presence, and range

validName ← calls a method to check if the data is of the correct length, type and presence

validSurname ← calls a method to check if the data is of the correct length, type and presence

IF (validName is true AND validSurname is true AND gender NOT BLANK validGrade is true AND validClass is true AND validPortfolio is true)

name ← name entered by the user

surname ← surname entered by the user

grade ← grade entered by the user

classPoint ← class entered by the user

catering ← false

photography ← false

firstAid ← false

av ← false

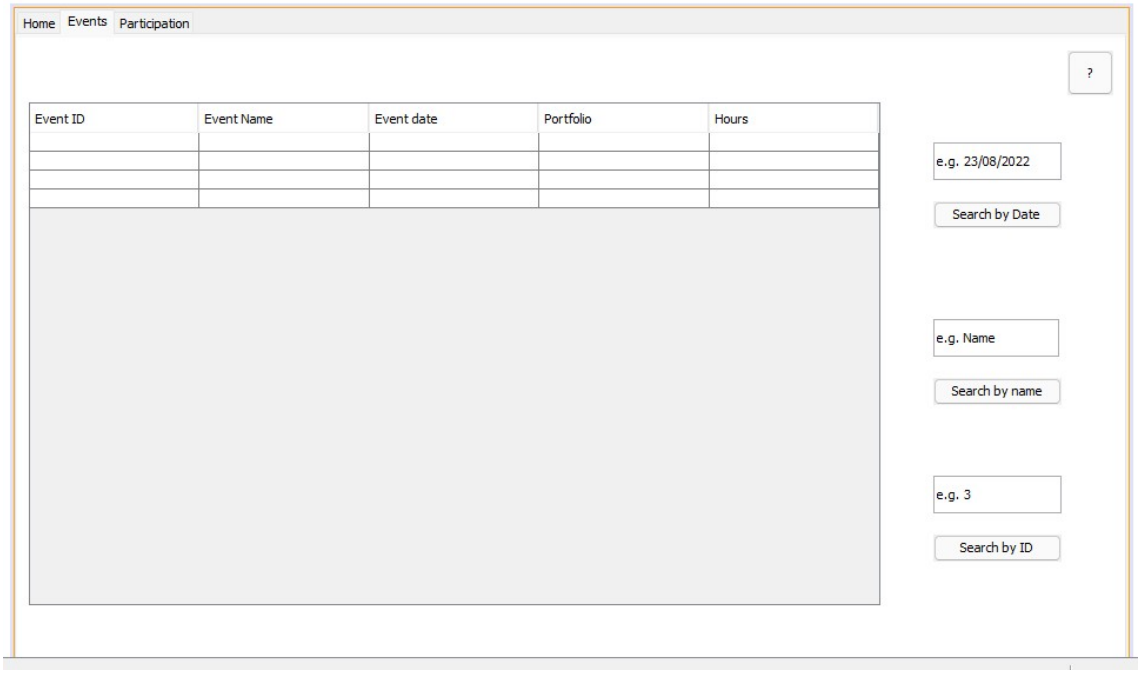
IF (cbxCatering1 Is Selected)

```
        catering ← true
    END IF
    IF (cbxPhotography1 is Selected)
        photography ← true
    END IF
    IF (cbxFirstAid1 is Selected)
        firstAid ← true
    END IF
    IF (cbxAV1 is Selected)
        av ← true
    END IF
Update Query ("UPDATE tblStudents SET firstName = '" + name+ "',Surname = '" + surname+
"', Grade = " + grade+  ",Class = " + classPoint + ", Gender = '" + gender + "', Catering = " +
catering + ", Photography = " + photography + ", AV = " + av + ", FirstAid = " + firstAid + "
WHERE StudentID = " + currentStu + ";");
        dlgEditStudent disposed
        tbdpStudent set visible
  ELSE
     Display "UNABLE TO UPDATE!"
   END ELSE
   IF (currentStu NOT -1)
      calls a method to show the student details on the home page
    END IF
```

2. Cancel button pressed:

```
dlgEditStudent disposed
```

Events:



1. Search by date button pressed

Connection to the database is made

Default table model set

tblEventStu remove all

dtm set row count to 0

tblEventStu set to the default table model

date ← dpkSearchDate get date entered by the user

rs ← SELECT * FROM tblEvents WHERE EventDate =#" + date +"#"

count ← 0

while rs has next

IF (rs.getBoolean("Catering") is true)

portfolio ← "Catering”

END IF

ELSE IF(rs.getBoolean("Photography") is true)

portfolio ← "Photography"

END ELSE IF

ELSE IF(rs.getBoolean("FirstAid") is true)

```
                portfolio ← "First Aid"
        END ELSE IF
           ELSE IF(rs.getBoolean("AV") is true)
              portfolio ← "AV"
        END ELSE IF
           c ←calls a method to check if the student does catering
           p ←calls a method to check if the student does photography
           a ←calls a method to check if the student does AV
           f ←calls a method to check if the student does first aid
           IF ((c is true AND portfolio = "Catering") OR (p is true AND portfolio = "Photography")
OR  (a is true AND portfolio = "AV") OR (f is true AND portfolio = “First Aid"))
              Add Row (new Object[] rs.getInt("EventID"),rs.getString("EventName"),
               rs.getInt("Hours"), rs.getDate("EventDate"), portfolio)
              count ← count + 1
           END IF
      END WHILE
              IF (count = 0)
               display "Entry not found
        END IF
        dpkSearchDate set text to “”
        model add Row
```

2. Search by name button pressed

```
Connection to the database is made
Default table model set
tblEventStu remove all
dtm set row count to 0
tblEventStu set to the default table model
     rs ← SELECT * FROM tblEvents WHERE EventName LIKE'*" +
txfSearchEventName.getText() +"*'"
     count ← 0
        while rs has next
```

```
        IF (rs.getBoolean("Catering") is true)
            portfolio ← "Catering”
    END IF
      ELSE IF(rs.getBoolean("Photography") is true)
                portfolio ← "Photography"
    END ELSE IF
      ELSE IF(rs.getBoolean("FirstAid") is true)
            portfolio ← "First Aid"
    END ELSE IF
       ELSE IF(rs.getBoolean("AV") is true)
          portfolio ← "AV"
    END ELSE IF
       c ←calls a method to check if the student does catering
       p ←calls a method to check if the student does photography
       a ←calls a method to check if the student does AV
       f ←calls a method to check if the student does first aid
       IF ((c is true AND portfolio = "Catering") OR (p is true AND portfolio = "Photography")
OR  (a is true AND portfolio = "AV") OR (f is true AND portfolio = “First Aid"))
          Add Row (new Object[] rs.getInt("EventID"),rs.getString("EventName"),
           rs.getInt("Hours"), rs.getDate("EventDate"), portfolio)
          count ← count + 1
       END IF
  END WHILE
            IF (count = 0)
             display "Entry not found
    END IF
    txfSearchEventName set text to “”
    model add Row
```

3. Search by ID button pressed

Connection to the database is made

Default table model set

```
tblEventStu remove all
dtm set row count to 0
tblEventStu set to the default table model
    rs ← SELECT * FROM tblEvents WHERE EventID =" + tfxSearchEventID.getText()
    count ← 0
     WHILE rs has next
          IF (rs.getBoolean("Catering") is true)
                portfolio ← "Catering”
   END IF
         ELSE IF(rs.getBoolean("Photography") is true)
                portfolio ← "Photography"
   END ELSE IF
         ELSE IF(rs.getBoolean("FirstAid") is true)
                portfolio ← "First Aid"
   END ELSE IF
          ELSE IF(rs.getBoolean("AV") is true)
             portfolio ← "AV"
   END ELSE IF
          c ←calls a method to check if the student does catering
          p ←calls a method to check if the student does photography
          a ←calls a method to check if the student does AV
          f ←calls a method to check if the student does first aid
          IF ((c is true AND portfolio = "Catering") OR (p is true AND portfolio =
"Photography") OR  (a is true AND portfolio = "AV") OR (f is true AND portfolio = “First Aid"))
             Add Row (new Object[] rs.getInt("EventID"),rs.getString("EventName"),
              rs.getInt("Hours"), rs.getDate("EventDate"), portfolio)
             count ← count + 1
          END IF
    END WHILE
           IF (count = 0)
```

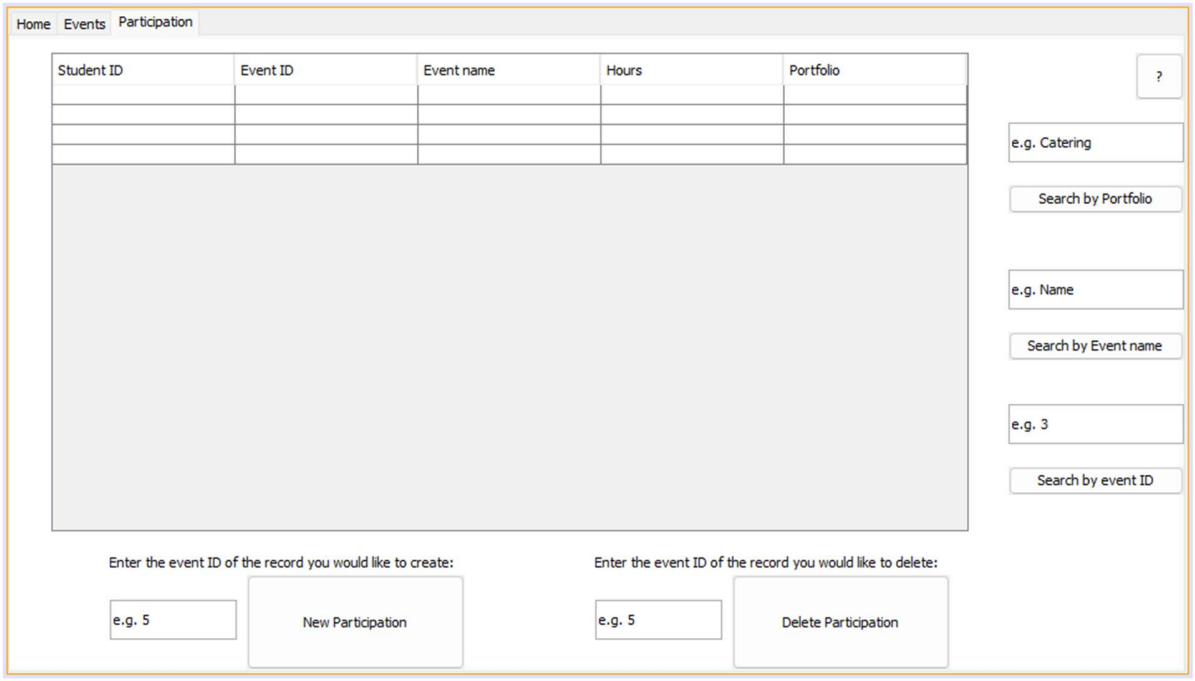
display "Entry not found

END IF

tfxSearchEventID set text to “”

model add Row

Participation:



1. Search by portfolio button pressed

Connection to the database is made

Default table model set

tblParticipation remove all

dtm set row count to 0

tblParticipation set to the default table model

rs ← SELECT DISTINCT tblStudents.StudentID, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND tblStudents.StudentID = " + currentStu

count ← 0

WHILE rs has next

portfolio ← ""

```
    IF (rs.getBoolean("Catering") is true AND txfSearchPartPortfolio = "Catering" OR
txfSearchPartPortfolio first character =  'c' OR txfSearchPartPortfolio first character = 'C')

            portfolio ← "Catering"

    END IF

    ELSE IF (rs.getBoolean("Photography") is true AND txfSearchPartPortfolio =
"Photography" OR txfSearchPartPortfolio first character = 'p' OR  txfSearchPartPortfolio first
character ='P')

            portfolio ← "Photography

    END ELSE IF

    ELSE IF (rs.getBoolean("FirstAid") is true AND (txfSearchPartPortfolio = ("First Aid") OR
txfSearchPartPortfolio first character = 'F' OR  txfSearchPartPortfolio first character = 'f'))

            portfolio ←  "First Aid”

    END ELSE IF

     ELSE IF(rs.getBoolean("AV") is true AND txfSearchPartPortfolio =  "AV" OR
txfSearchPartPortfolio first character = 'a' OR  txfSearchPartPortfolio first character = 'A')

            portfolio ← "AV"

    END ELSE IF

        c ←calls a method to check if the student does catering

        p ←calls a method to check if the student does photography

        a ←calls a method to check if the student does AV

        f ←calls a method to check if the student does first aid

        IF ((c is true AND portfolio = "Catering") OR (p is true AND portfolio =
"Photography") OR  (a is true AND portfolio = "AV") OR (f is true AND portfolio = “First Aid"))

            model.addRow(new Object (rs.getInt("StudentID"), rs.getInt("EventID"),
rs.getString("EventName"),rs.getInt("Hours"), portfolio))

            count ← count + 1

          END IF

     END WHILE

       IF (count = 0)

          Display “entry not found”

       END IF

       txfSearchPartPortfolio set Text ""
```

model add Rows to the table

2. Search by event name button pressed

Connection to the database is made

Default table model set

tblParticipation remove all

dtm set row count to 0

tblParticipation set to the default table model

rs ← SELECT DISTINCT tblStudents.StudentID, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND EventName LIKE '*" + txfSearchPartName.getText() + "*' AND tblStudents.StudentID = " + currentStu

count ← 0

WHILE rs has next

portfolio ← ""

IF (rs.getBoolean("Catering") is true)

portfolio ← "Catering”

END IF

ELSE IF(rs.getBoolean("Photography") is true)

portfolio ← "Photography"

END ELSE IF

ELSE IF(rs.getBoolean("FirstAid") is true)

portfolio ← "First Aid"

END ELSE IF

ELSE IF(rs.getBoolean("AV") is true)

portfolio ← "AV"

END ELSE IF

c ←calls a method to check if the student does catering

p ←calls a method to check if the student does photography

a ←calls a method to check if the student does AV

f ←calls a method to check if the student does first aid

IF ((c is true AND portfolio = "Catering") OR (p is true AND portfolio = "Photography") OR (a is true AND portfolio = "AV") OR (f is true AND portfolio = “First Aid"))

model.addRow(new Object (rs.getInt("StudentID"), rs.getInt("EventID"), rs.getString("EventName"),rs.getInt("Hours"), portfolio))

count ← count + 1

END IF

END WHILE

IF (count = 0)

Display “entry not found”

END IF

txfSearchPartName set Text ""

model add Rows to the table

3. Search by event ID button pressed

Connection to the database is made

Default table model set

tblParticipation remove all

dtm set row count to 0

tblParticipation set to the default table model

rs ← SELECT DISTINCT tblStudents.StudentID, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND tblEvents.EventID = " + Integer.parseInt(txfSearchPartID.getText()) + " AND tblStudents.StudentID = " + currentStu

count ← 0

WHILE rs has next

portfolio ← ""

IF (rs.getBoolean("Catering") is true)

portfolio ← "Catering”

END IF

ELSE IF(rs.getBoolean("Photography") is true)

portfolio ← "Photography"

END ELSE IF

ELSE IF(rs.getBoolean("FirstAid") is true)

portfolio ← "First Aid"

END ELSE IF

ELSE IF(rs.getBoolean("AV") is true)

portfolio ← "AV"

END ELSE IF

c ←calls a method to check if the student does catering

p ←calls a method to check if the student does photography

a ←calls a method to check if the student does AV

f ←calls a method to check if the student does first aid

IF ((c is true AND portfolio = "Catering") OR (p is true AND portfolio = "Photography") OR (a is true AND portfolio = "AV") OR (f is true AND portfolio = “First Aid"))

model.addRow(new Object (rs.getInt("StudentID"), rs.getInt("EventID"), rs.getString("EventName"),rs.getInt("Hours"), portfolio))

count ← count + 1

END IF

END WHILE

IF (count = 0)

Display “entry not found”

END IF

txfSearchPartID set Text ""

model add Rows to the table

4. New participation button pressed

Event ← txfEventIDCreate text from user

add(Event, currentStu)

display “successful entry”

txfEventIDCreate set Text to "e.g 5

calls a method to refresh the table for the user

5. Delete participation button pressed

Connection to the database is made

Default table model set

```
tblParticipation remove all

dtm set row count to 0

tblParticipation set to the default table model

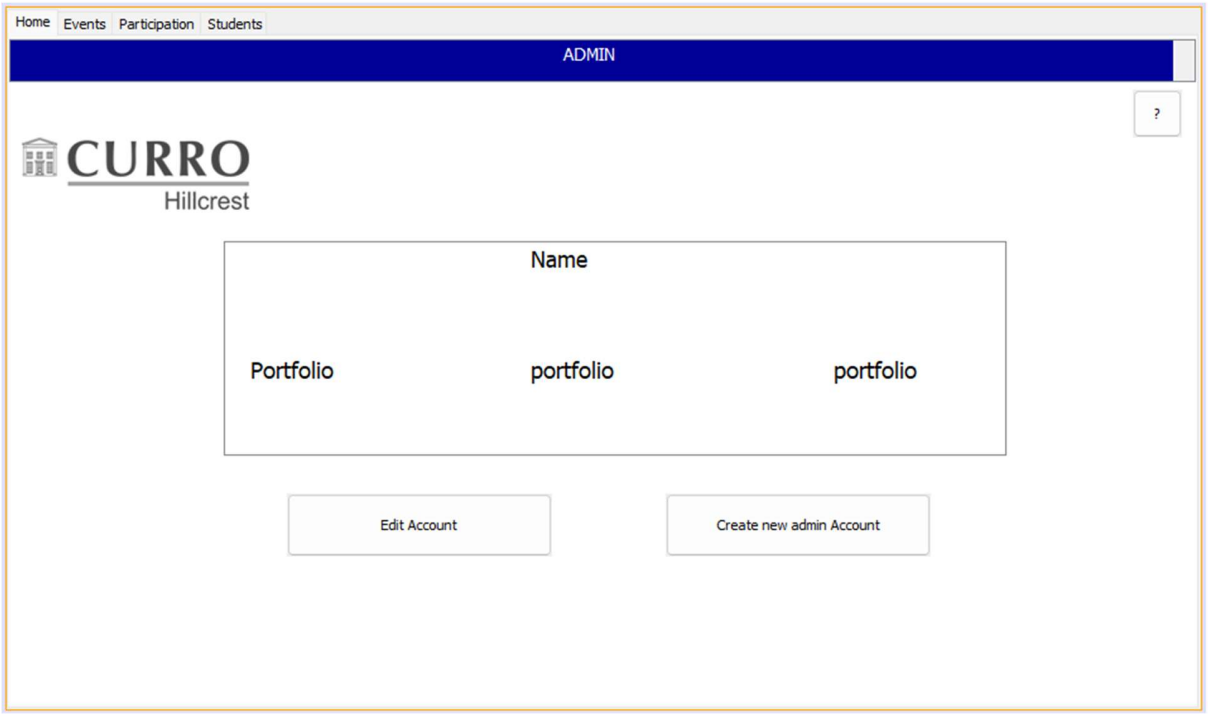
rs ← SELECT DISTINCT tblStudents.StudentID, tblEvents.EventID, tblEvents.EventName,
tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid
FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID =
tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
tblEvents.EventID = " + Integer.parseInt(txfEventIDDelete.getText()) + " AND
tblStudents.StudentID = " + currentStu

count ← 0

WHILE rs has next
        portfolio ← ""
    IF (rs.getBoolean("Catering") is true)
            portfolio ← "Catering”
    END IF
        ELSE IF(rs.getBoolean("Photography") is true)
            portfolio ← "Photography"
    END ELSE IF
        ELSE IF(rs.getBoolean("FirstAid") is true)
            portfolio ← "First Aid"
    END ELSE IF
        ELSE IF(rs.getBoolean("AV") is true)
            portfolio ← "AV"
    END ELSE IF
        c ←calls a method to check if the student does catering
        p ←calls a method to check if the student does photography
        a ←calls a method to check if the student does AV
        f ←calls a method to check if the student does first aid
        IF ((c is true AND portfolio = "Catering") OR (p is true AND portfolio =
"Photography") OR  (a is true AND portfolio = "AV") OR (f is true AND portfolio = “First Aid"))
            s ← rs.getInt("StudentID”)
            e ← rs.getInt("EventID")
```

```
            en ← rs.getString("EventName")
            h ← rs.getInt("Hours")
           add Row(new Object[]{s, e, en, h, portfolio})
            answer ← choice display “Are you sure your want to delete this record?
            IF (answer = NO)
               Display delete canceled
          END IF
            ELSE IF (answer = YES)
               Delete Entry (txfEventIDDelete ,currentStu)
               Calls a method to reload the participation table
            END ELSE IF
          END IF
END WHILE
   IF (count = 0)
      Display “entry not found”
   END IF
   model add Rows to the table
```

Admin home page:



1. Edit account button pressed:

Calls a method which will clear all error labels

dlgEditAdmin set enabled

dlgEditAdmin set Size (900,600)

dlgEditAdmin set Visible

txfEditAdminName set Text("")

txfEditAdminSurname set Text("")

rs ← SELECT * FROM tblAdmins WHERE AdminID = " + currentAdmin

WHILE rs has next

IF (rs.getBoolean("Catering") is true)

cbxCateringAdmin set to selected

END IF

IF (rs.getBoolean("Photography") is true)

cbxPhotographyAdmin set to selected

END IF

IF (rs.getBoolean("First Aid") is true)

```
        cbxFirstAidAdmin set to selected
    END IF
    IF (rs.getBoolean("AV") is true)
        cbxAVAdmin set to selected
    END IF
    txfEditAdminName set Text to (rs.getString("AdminName"))
    txfEditAdminSurname set Text to (rs.getString("AdminSurname"))
END WHILE
```

2. Create new admin account button pressed:

Calls a method to clear all the error lables

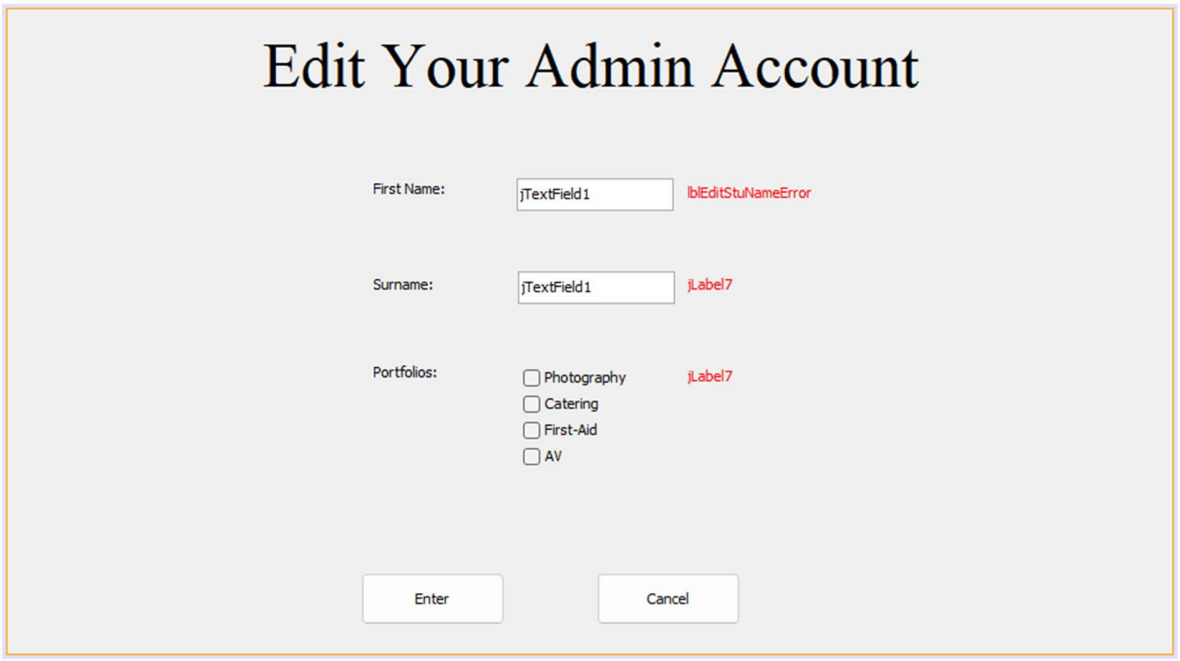
```
dlgNewAdmin set to Enabled
dlgNewAdmin set Size (900,600)
dlgNewAdmin set Visible
txfNewAdminName set Text("")
txfNewAdminUsername set Text("")
txfNewAdminPassword set Text("")
txfNewAdminConfirmPassword set Text("")
```

3. Help button pressed:

pnlHelp set Visible

```
pnlHomeAdmin set Visible
tbdpStudent disposed
tbdpAdmin disposed
help ← 3
```

Edit admin account page:



1. Enter button pressed:

Calls a method to clear all error labels

validPortfolio ← calls a method to check that a valid portfolio selection has been made

validName ← calls methods to perform presence, length and type checks

validSurname ← calls methods to perform presence, length and type checks

IF ((validName is true) AND (validSurname is true) AND (validPortfolio is true));

name ← txfEditAdminName

surname ← txfEditAdminSurname

catering ← false

photography ← false

firstAid ← false

av ← false

IF (cbxCateringAdmin is Selected)

catering ← true

END IF

IF (cbxPhotographyAdmin is Selected)

photography ← true

END IF

IF (cbxFirstAidAdmin is Selected)

firstAid ← true

END IF

IF (cbxAVAdmin is Selected)

av ← true

END IF

Dbman update Query ("UPDATE tblAdmins SET AdminName = '" + name+ "',AdminSurname = '" + surname+ "', Catering = " + catering + ", Photography = " + photography + ", AV = " + av + ", [First Aid] = " + firstAid + " WHERE AdminID = " + currentAdmin + ";")

dlgEditAdmin disposed

tbdpAdmin set visible

END IF

IF (currentAdmin is NOT -1)

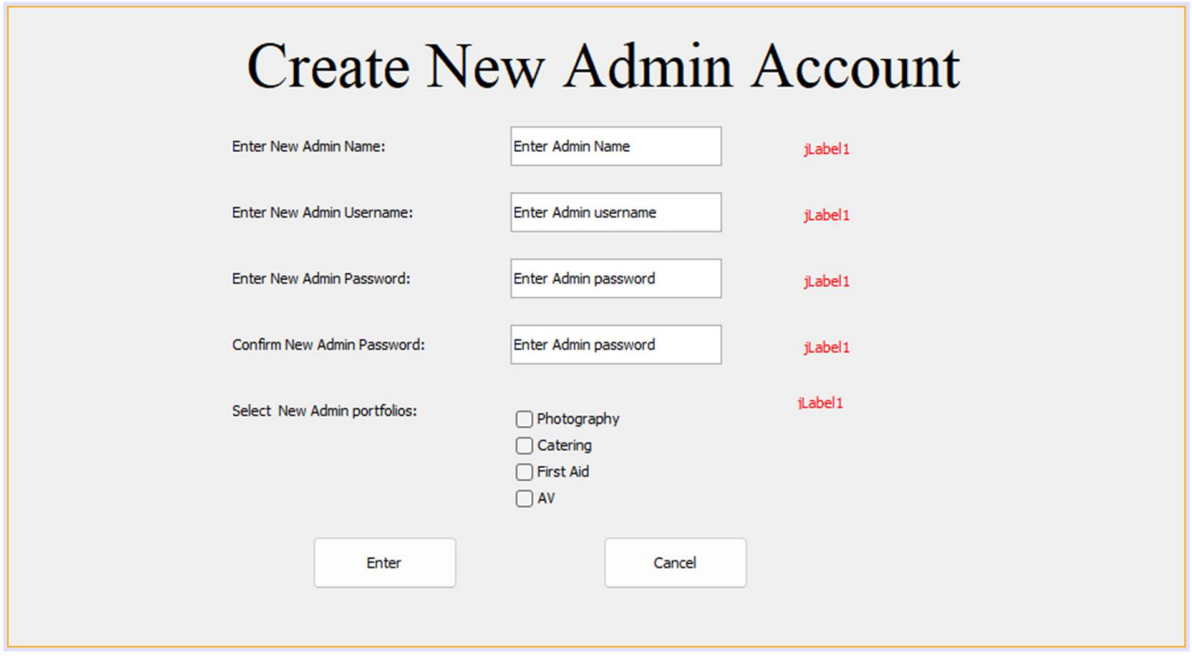
calls a method to show the admin details on the home page

END IF

2. Cancel button pressed:

dlgEditAdmin disposed

New Admin Account:



1. Enter button pressed:

Calls a method to clear all error labels

outFile ← new Print Writer

password ← false

IF (txfNewAdminPassword = txfNewAdminConfirmPassword)

password ← true

END IF

ELSE

lblNewAdminConfirmPasswordError set Text ("Passwords Do Not Match")

END ELSE

validPortfolio ← calls a method to check that a valid portfolio selection has been made

validUserName ← calls methods to perform presence, length and type checks

validPassword ← calls methods to perform presence and length

boolean validName ← calls methods to perform presence, length and type checks as well as a check to ensure that a full name has been entered

IF ((validUserName is true) AND (validPassword is true) AND (validName is true) AND (validPortfolio is true) AND (password is true))

name ← txfNewAdminName

catering ← false

photography ← false

firstAid ← false

av ← false

lName ← anything after the last space

fName ← anything before the last space

IF (cbxNewAdminCatering is Selected)

catering ← true

END IF

IF (cbxNewAdminPhotography is Selected)

photography ← true

END IF

IF (cbxNewAdminFirstAid is Selected)

firstAid ← true

END IF

IF (cbxNewAdminAv is Selected)

av ← true

END IF

add (fName,lName , catering, photography, av, firstAid)

line ← (am.getSize()-1) + "#" + txfNewAdminUsername.getText() + "#" + txfNewAdminPassword.getText() + "#Admin"

outFile add line to text file

display “new admin created”

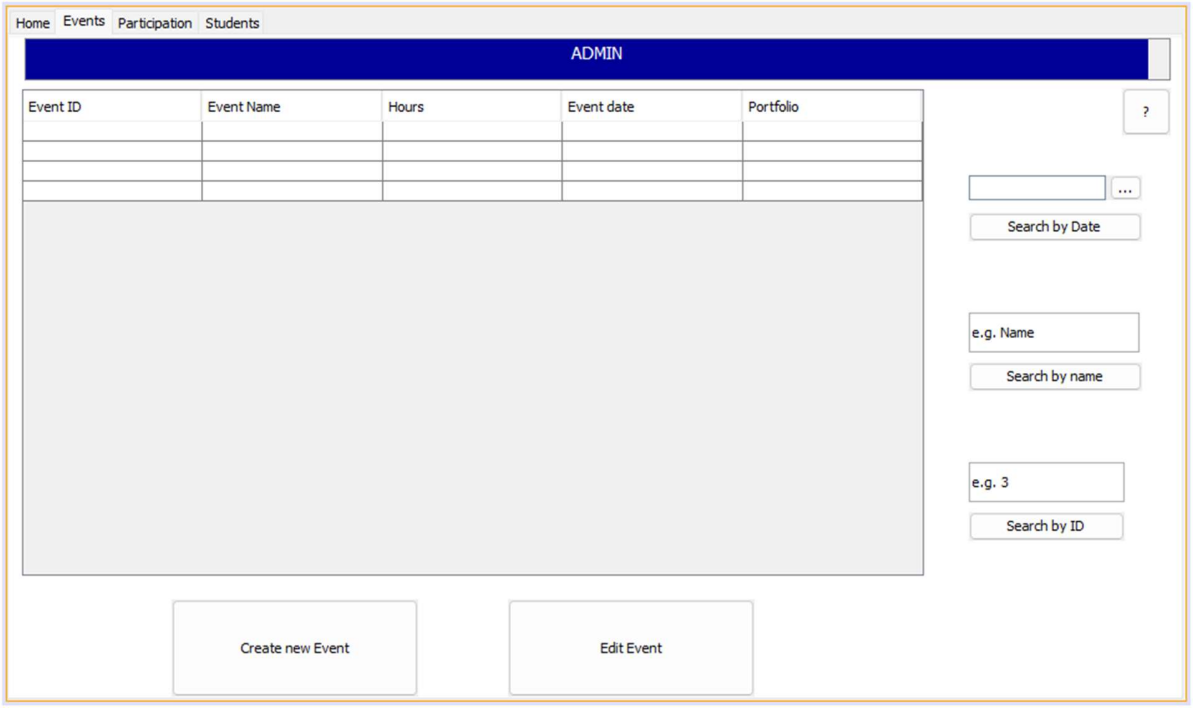
dlgNewAdmin disposed

END IF

2. Cancel button pressed:

dlgNewAdmin disposed

Admin Events page:



1. Search by date button pressed:

Connection to the database is made

Default table model set

```
tblEventsAdmin remove all
dtm set row count to 0
tblEventsAdmin set to the default table model
date ← dpkSearch
rs ← “SELECT * FROM tblEvents WHERE EventDate =#" + date +"#"
count ← 0
WHILE rs has next
    portfolio ← ""
    IF (rs.getBoolean("Catering") is true)
       portfolio ← "Catering"
    END IF
    ELSE IF (rs.getBoolean("Photography") is true)
       portfolio ← "Photography"
    END ELSE IF
    ELSE IF (rs.getBoolean("FirstAid") is true)
       portfolio ← "First Aid"
    END ELSE IF
    ELSE IF (rs.getBoolean("AV") is true)
       portfolio ← "AV"
    END ELSE IF
    c ← calls a method to check if the admin oversees catering
    p ← calls a method to check if the admin oversees photography
    a ← calls a method to check if the admin oversees AV
    f ← calls a method to check if the admin oversees first aid
    IF ((c is true AND portfolio = ("Catering")) OR (p is true AND  portfolio =
("Photography")) OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = "First Aid"))
       count ← count +1
       add Row(rs.getInt("EventID"), rs.getString("EventName"),
       rs.getInt("Hours"), rs.getDate("EventDate"), portfolio)
    END IF
```

```
    END WHILE
      IF (count = 0)
        Displays entry not found
      END IF
      dpkSearch set Text("")
      Add row to the table
```

2. Search by name button pressed:

```
Connection to the database is made
Default table model set
      tblEventsAdmin remove all
      dtm set row count to 0
      tblEventsAdmin set to the default table model
    rs ← "SELECT * FROM tblEvents WHERE EventName LIKE '*" +
(txfSearchName.getText()) + "*'")
      count ← 0
    WHILE rs has next
        portfolio ← ""
        IF (rs.getBoolean("Catering") is true)
          portfolio ← "Catering"
        END IF
        ELSE IF (rs.getBoolean("Photography") is true)
          portfolio ← "Photography"
        END ELSE IF
        ELSE IF (rs.getBoolean("FirstAid") is true)
          portfolio ← "First Aid"
        END ELSE IF
        ELSE IF (rs.getBoolean("AV") is true)
          portfolio ← "AV"
        END ELSE IF
        c ← calls a method to check if the admin oversees catering
        p ← calls a method to check if the admin oversees photography
```

```
        a ← calls a method to check if the admin oversees AV
        f ← calls a method to check if the admin oversees first aid
        IF ((c is true AND portfolio = ("Catering")) OR (p is true AND  portfolio =
("Photography")) OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = "First Aid"))
            count ← count +1
            add Row(rs.getInt("EventID"), rs.getString("EventName"),
            rs.getInt("Hours"), rs.getDate("EventDate"), portfolio)
        END IF
    END WHILE
    IF (count = 0)
        Displays entry not found
    END IF
    txfSearchName set Text("")
    Add row to the table
```

3. Search by ID button pressed:

```
Connection to the database is made
Default table model set
    tblEventsAdmin remove all
    dtm set row count to 0
    tblEventsAdmin set to the default table model
     rs ← SELECT * FROM tblEvents WHERE EventID =" +txfSearchID.getText()
     count ← 0
    WHILE rs has next
        portfolio ← ""
        IF (rs.getBoolean("Catering") is true)
            portfolio ← "Catering"
        END IF
        ELSE IF (rs.getBoolean("Photography") is true)
            portfolio ← "Photography"
        END ELSE IF
        ELSE IF (rs.getBoolean("FirstAid") is true)
```

```
            portfolio ← "First Aid"
        END ELSE IF
        ELSE IF (rs.getBoolean("AV") is true)
            portfolio ← "AV"
        END ELSE IF
        c ← calls a method to check if the admin oversees catering
        p ← calls a method to check if the admin oversees photography
        a ← calls a method to check if the admin oversees AV
        f ← calls a method to check if the admin oversees first aid
        IF ((c is true AND portfolio = ("Catering")) OR (p is true AND  portfolio =
("Photography")) OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = "First
Aid"))
            count ← count +1
            add Row(rs.getInt("EventID"), rs.getString("EventName"),
            rs.getInt("Hours"), rs.getDate("EventDate"), portfolio)
        END IF
    END WHILE
      IF (count = 0)
        Displays entry not found
      END IF
      txfSearchName set Text("")
     Add row to the table
```

4. Create new event button pressed:

Calls a method to clear all error labels

```
dlgEvents set to enables
dlgEvents set Size (900,600)
dlgEvents set Visible
rbnNewEventCatering set Visible
rbnNewEventPhotography set Visible
rbnNewEventAV set Visible
rbnNewEventFirstAid set Visible
```

IF ((calls a method to check if the admin oversees catering) is false)

rbnNewEventCatering disposed

END IF

IF (calls a method to check if the admin oversees photography) is false)

rbnNewEventPhotography disposed

END IF

IF (calls a method to check if the admin oversees AV) is false)

rbnNewEventAV disposed

END IF

IF (calls a method to check if the admin oversees first aid) is false)

rbnNewEventFirstAid disposed

END IF

5. Edit event button pressed:

Calls a method to clear all error labels

dlgEditEvent set to Enabled

dlgEditEvent set Size(900,600)

dlgEditEvent set Visible

txfEditEventName disposed

txfEditEventHours disposed

dpkEditDate disposed

rbnEditEventCatering disposed

rbnEditEventFirstAid disposed

rbnEditEventPhotography disposed

rbnEditEventAV disposed

btnEditEventEnter disposed

lblEditEventName disposed

lblEditEventHours disposed

lblEditEventDate disposed

lblEditEventPortfolio disposed

btnEditEventSearch set Visible

6. Help button pressed:

pnlHelp set Visible

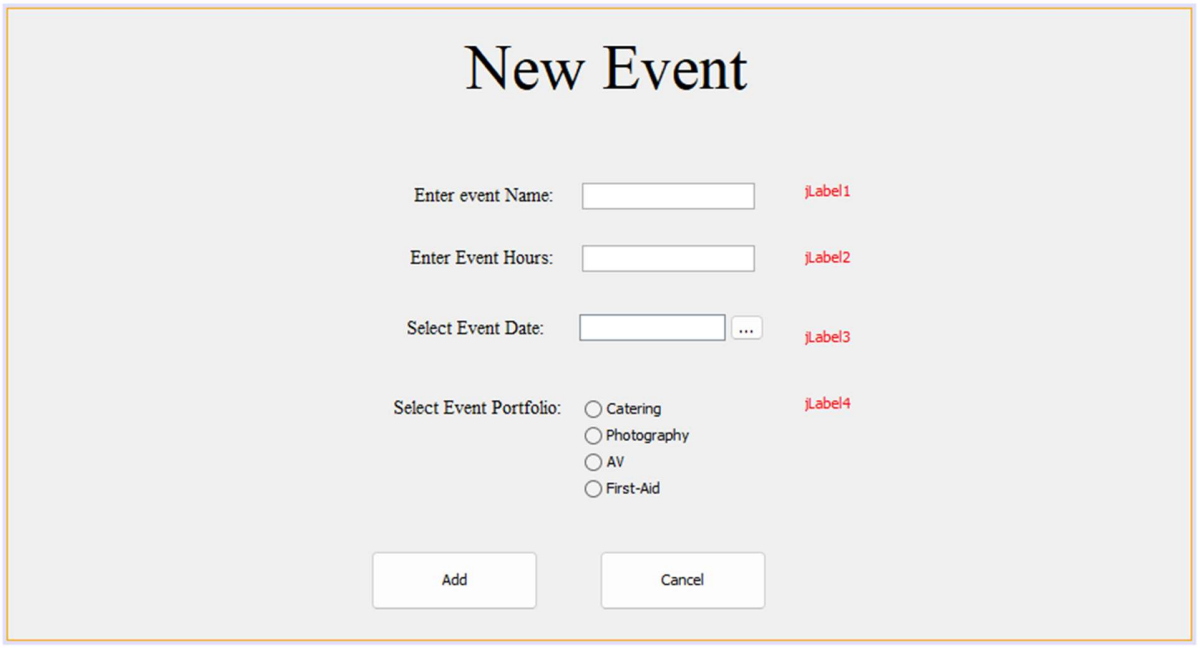
pnlEvents1 disposed

tbdpStudent disposed

tbdpAdmin disposed

help ← 4

Create new event:



1. Add button pressed

Calls a method to clear all error labels

validName ← calls methods to perform Presence, Length and Type checks

validDate ← calls methods to perform presence checks

catering ← false

av ← false

photography ← false

firstAid ← false

selected ← false

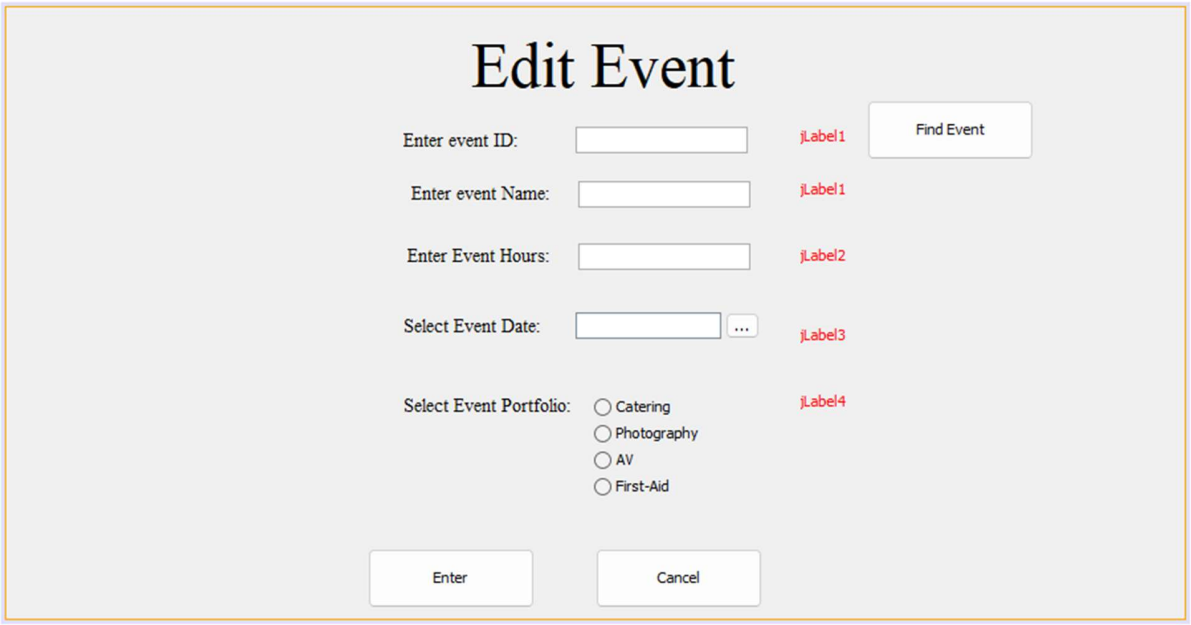
IF (rbnNewEventCatering is Selected)

catering ← true

```
    selected ← true
  END IF
  ELSE IF (rbnNewEventAV is Selected)
            av ←true
    selected ← true
  END ELSE IF
  ELSE IF (rbnNewEventPhotography is Selected)
       photography ← true
    selected ← true
  END ELSE IF
  ELSE IF (rbnNewEventFirstAid is Selected)
      firstAid ← true
    selected ← true
  END IF
 ELSE
    lblNewPortfolioError set Text("Select a portfolio")
  END ELSE
  hours ← checkHours(txfNewEventHours, lblNewHoursError)
  h ← Double (hours)
  IF (validName is true AND validDate is true AND hours is NOT null AND selected is true)
        name ← txfNewEventName
        dateEvent ← dpkDate
        add event (name,h ,dateEvent, catering, photography, firstAid, av)
        dlgEvents disposed
        display “event entered”
        calls a method to refresh the events table
```

2. Cancel button pressed:
   dlgEvents disposed

Edit event:



1. Find event button pressed:

IF (txfEditEventID. IS NOT BLANK)

Event ← txfEditEventID

txfEditEventName set Visible

txfEditEventHours set Visible

dpkEditDate set Visible

rbnEditEventCatering set Visible

rbnEditEventPhotography set Visible

rbnEditEventAV set Visible

rbnEditEventFirstAid set Visible

IF ((calls a method to check if the admin oversees catering) is false)

rbnEditEventCatering is disposed

END IF

IF ((calls a method to check if the admin oversees photography) is false)

rbnEditEventPhotography disposed

END IF

IF ((calls a method to check if the admin oversees AV) is false)

rbnEditEventAV disposed

```
END IF
IF ((calls a method to check if the admin oversees FirstAid) is false)
   rbnEditEventFirstAid disposed
END IF
btnEditEventEnter set Visible
lblEditEventName set Visible
lblEditEventHours set Visible
lblEditEventDate set Visible
lblEditEventPortfolio set Visible
rs ← SELECT * FROM tblEvents WHERE EventID =" +txfEditEventID.getText
WHILE rs has next
   portfolio ← ""
        IF (rs.getBoolean("Catering") is true)
          portfolio ← "Catering"
        rbnEditEventCatering set Selected
       END IF
       ELSE IF (rs.getBoolean("Photography") is true)
          portfolio ← "Photography"
        rbnEditEventPhotography set Selected
       END ELSE IF
       ELSE IF (rs.getBoolean("FirstAid") is true)
          portfolio ← "First Aid"
        rbnEditEventFirstAid set Selected
       END ELSE IF
       ELSE IF (rs.getBoolean("AV") is true)
          portfolio ← "AV"
        rbnEditEventAV set Selected
       END ELSE IF
       c ← calls a method to check if the admin oversees catering
       p ← calls a method to check if the admin oversees photography
```

```
            a ← calls a method to check if the admin oversees AV
            f ← calls a method to check if the admin oversees first aid
    IF ((c is true AND portfolio = ("Catering")) OR (p is true AND  portfolio = ("Photography"))
    OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = "First Aid"))
    txfEditEventID.setText(rs.getString("EventID"));//sets the text field to the event ID
            txfEditEventName set Text (rs.getString("EventName"))
            txfEditEventHours set Text (rs.getString("Hours"))
            Date ← rs.getDate("EventDate")
             dpkEditDate set Date(Date)
        END IF
  END WHILE
    btnEditEventSearch disposed
  END IF
  ELSE
    lblEditEventID set Text ("Enter An Event ID)
  END ELSE
```

2. Enter button pressed:

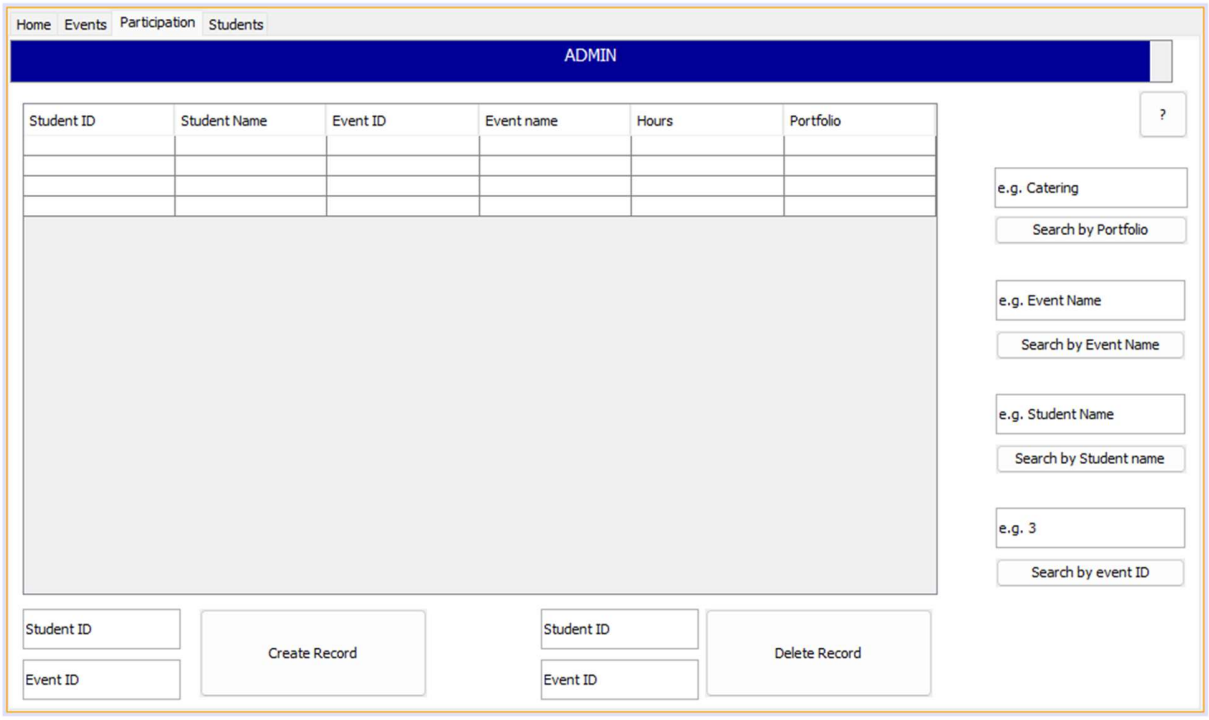
```
Calls a method to clear all error labels
      validName ← calls methods to perform Presence, Length and Type checks
      validDate ← calls methods to perform presence checks
      catering ← false
      av ← false
      photography ← false
      firstAid ← false
      selected ← false
      IF (rbnEditEventCatering is Selected)
         catering ← true
         selected ← true
      END IF
      ELSE IF (rbnEditEventAV is Selected)
            av ←true
```

```
selected ← true
        END ELSE IF
        ELSE IF (rbnEditEventPhotography is Selected)
             photography ← true
selected ← true
        END ELSE IF
        ELSE IF (r bnEditEventFirstAid is Selected)
            firstAid ← true
selected ← true
        END IF
       ELSE
          lblNewPortfolioError set Text("Select a portfolio")
        END ELSE
        hours ← checkHours(txfEditEventHours, lblEditHoursError)
        h ← Double (hours)
        IF (validName is true AND validDate is true AND hours is NOT null AND selected is
true)
              name ← txfEditEventName
              dateEvent ← dpkEditDate
              sql ← ("UPDATE tblEvents SET EventName = '" + name+ "',Hours = " + hours+ ",
EventDate = #" + dateEvent + "#, Catering = " + catering + ", Photography = " + photography
+  ", FirstAid = " + firstAid + ", AV = " + av + " WHERE EventID = " + txfEditEventID.getText()
+ ";");
              Dbman update Query(sql)
               dlgEditEvent disposed
              display "Event successfully Updated"
              calls a method to refresh the events table
```

3. Cancel button pressed:

```
dlgEditEvent disposed
```

Admin participation page:



1. Search by portfolio button pressed:

Connection to the database is made

Default table model set

tblParticipationAdmin remove all

dtm set row count to 0

tblParticipationAdmin set to the default table model

rs ←"SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID "

count ← 0

while rs next

portfolio ← “”

IF (rs.getBoolean("Catering") is true AND txfSearchPortfolioPart = "Catering")

portfolio ← "Catering"

END IF

IF (rs.getBoolean("Photography") is true AND txfSearchPortfolioPart = "Photography")

```
            portfolio ← "Photography"
        END IF
        IF (rs.getBoolean("FirstAid") is true AND  txfSearchPortfolioPart = "First Aid")
            portfolio ← "First Aid"
        END IF
        IF (rs.getBoolean("AV") is true AND txfSearchPortfolioPart = "AV")
            portfolio ← "AV"
        END IF
        c ←  calls a method which checks if the admin oversees catering
        p ← calls a method which checks if the admin oversees photography
        a ← calls a method which checks if the admin oversees AV
        f ← calls a method which checks if the admin oversees first aid
        IF ((c is true AND portfolio = ("Catering")) OR  (p is true AND portfolio = ("Photography")) OR (a
is true AND portfolio = ("AV")) OR (f is true AND portfolio = ("First Aid")))
                name ← rs.getString("firstName") + " " + rs.getString("surname")
                add Row (rs.getInt("StudentID"), name, rs.getInt("EventID"), rs.getString("EventName"),
            rs.getInt("Hours"), portfolio)
                count ← count +1
        END IF
    END WHILE
      IF (count = 0)
        Display “entry not found”
      END IF
      txfSearchPortfolioPart set Text to ("")
      model add Row
```

2. Search by event name button pressed:

Connection to the database is made

Default table model set

tblParticipationAdmin remove all

dtm set row count to 0

```
tblParticipationAdmin set to the default table model

rs ← SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering,
tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents,
tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID AND EventName LIKE '*" +
txfSearchEventNamePart.getText() + "*'"

 count ← 0

 while rs next

        portfolio ← “”

        IF (rs.getBoolean("Catering") is true)

          portfolio ← "Catering"

        END IF

        IF (rs.getBoolean("Photography") is true)

          portfolio ← "Photography"

        END IF

        IF (rs.getBoolean("FirstAid") is true)

          portfolio ← "First Aid"

        END IF

        IF (rs.getBoolean("AV") is true)

          portfolio ← "AV"

        END IF

        c ←  calls a method which checks if the admin oversees catering

        p ← calls a method which checks if the admin oversees photography

        a ← calls a method which checks if the admin oversees AV

        f ← calls a method which checks if the admin oversees first aid

        IF ((c is true AND portfolio = ("Catering")) OR  (p is true AND portfolio = ("Photography"))
OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = ("First Aid")))

            name ← rs.getString("firstName") + " " + rs.getString("surname")

            add Row (rs.getInt("StudentID"), name, rs.getInt("EventID"), rs.getString("EventName"),

           rs.getInt("Hours"), portfolio)
```

```
            count ← count +1
        END IF
    END WHILE
    IF (count = 0)
        Display “entry not found”
    END IF
    txfSearchEventNamePart set Text to ("")
    models add Row
```

3. Search by student name button pressed:

Connection to the database is made

Default table model set

tblParticipationAdmin remove all

dtm set row count to 0

tblParticipationAdmin set to the default table model

fullname ← txfSearchStudentNamePart

lName ← ""

fName ← ""

IF (fullname contains(“”)

lName ← anything after the last space

fName ← everything before the last space

END IF

```
rs ←"SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering,
tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents,
tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID  ")
 count ← 0
 while rs next
        portfolio ← “”
        IF (rs.getBoolean("Catering") is true)
          portfolio ← "Catering"
```

```
            END IF
            IF (rs.getBoolean("Photography") is true)
               portfolio ← "Photography"
            END IF
            IF (rs.getBoolean("FirstAid") is true)
               portfolio ← "First Aid"
            END IF
            IF (rs.getBoolean("AV") is true)
               portfolio ← "AV"
            END IF
            c ←  calls a method which checks if the admin oversees catering
            p ← calls a method which checks if the admin oversees photography
            a ← calls a method which checks if the admin oversees AV
            f ← calls a method which checks if the admin oversees first aid
            IF ((c is true AND portfolio = ("Catering")) OR  (p is true AND portfolio = ("Photography"))
OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = ("First Aid")))
                name ← rs.getString("firstName") + " " + rs.getString("surname")
                add Row (rs.getInt("StudentID"), name, rs.getInt("EventID"), rs.getString("EventName"),
               rs.getInt("Hours"), portfolio)
                count ← count +1
            END IF
        END WHILE
        IF (count = 0)
           Display “entry not found”
        END IF
        txfSearchEventNamePart set Text to ("")
        models add Row
```

4. Search by event ID button pressed:

Connection to the database is made

Default table model set

tblParticipationAdmin remove all

dtm set row count to 0

tblParticipationAdmin set to the default table model

rs ← SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND EventID = '' + txfSearchEventIDPart.getText()

count ← 0

while rs next

portfolio ← “”

IF (rs.getBoolean("Catering") is true)

portfolio ← "Catering"

END IF

IF (rs.getBoolean("Photography") is true)

portfolio ← "Photography"

END IF

IF (rs.getBoolean("FirstAid") is true)

portfolio ← "First Aid"

END IF

IF (rs.getBoolean("AV") is true)

portfolio ← "AV"

END IF

c ← calls a method which checks if the admin oversees catering

p ← calls a method which checks if the admin oversees photography

a ← calls a method which checks if the admin oversees AV

f ← calls a method which checks if the admin oversees first aid

IF ((c is true AND portfolio = ("Catering")) OR (p is true AND portfolio = ("Photography")) OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = ("First Aid")))

name ← rs.getString("firstName") + " " + rs.getString("surname")

add Row (rs.getInt("StudentID"), name, rs.getInt("EventID"), rs.getString("EventName"), rs.getInt("Hours"), portfolio)

count ← count +1

END IF

END WHILE

IF (count = 0)

Display “entry not found”

END IF

txfSearchEventNamePart set Text to ("")

models add Row

5. Create record button pressed:

Event ← txfCreatePartEventID

Add (Event, txfCreatePartStuID)

Display “successful entry)

Calls a method which re-loads the admin participation table

6. Delete record button pressed:

Connection to the database is made

Default table model set

tblParticipationAdmin remove all

dtm set row count to 0

tblParticipationAdmin set to the default table model

rs ← SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND EventID = " + txfSearchEventIDPart.getText()

count ← 0

while rs next

```
portfolio ← “”
IF (rs.getBoolean("Catering") is true)
   portfolio ← "Catering"
END IF
IF (rs.getBoolean("Photography") is true)
   portfolio ← "Photography"
END IF
IF (rs.getBoolean("FirstAid") is true)
   portfolio ← "First Aid"
END IF
IF (rs.getBoolean("AV") is true)
   portfolio ← "AV"
END IF
c ←  calls a method which checks if the admin oversees catering
p ← calls a method which checks if the admin oversees photography
a ← calls a method which checks if the admin oversees AV
f ← calls a method which checks if the admin oversees first aid
IF ((c is true AND portfolio = ("Catering")) OR  (p is true AND portfolio = ("Photography")) OR (a is true AND portfolio = ("AV")) OR (f is true AND portfolio = ("First Aid")))
s ← rs.getInt("StudentID")
name ← rs.getString("firstname") + " " + rs.getString("surname”)
e ← rs.getInt("EventID")
en ← rs.getString("EventName")
h ← rs.getInt("Hours")
add Row (s,name,e,en,h, portfolio)
answer ← displays "Are you sure you want to delete this record?\n" + s + "-" + name + "-" + e + "-"+ en +"-"+h, "DELETE PARTICIPATION RECORD"
   IF (answer =NO)
   Displays “delete canceled”
```

END IF

ELSE IF (answer = YES)

Delete entry ((txfDeleteEventID), (txfDeleteStuID))

END ELSE IF

END IF

END WHILE

txfDeleteEventID set Text ("Enter Event ID")

txfDeleteStuID set Text ("Enter Student ID")

model add Row (new Object[]{})

calls a method which re-loads the admin participation table

7. Help button pressed:

pnlHelp set Visible

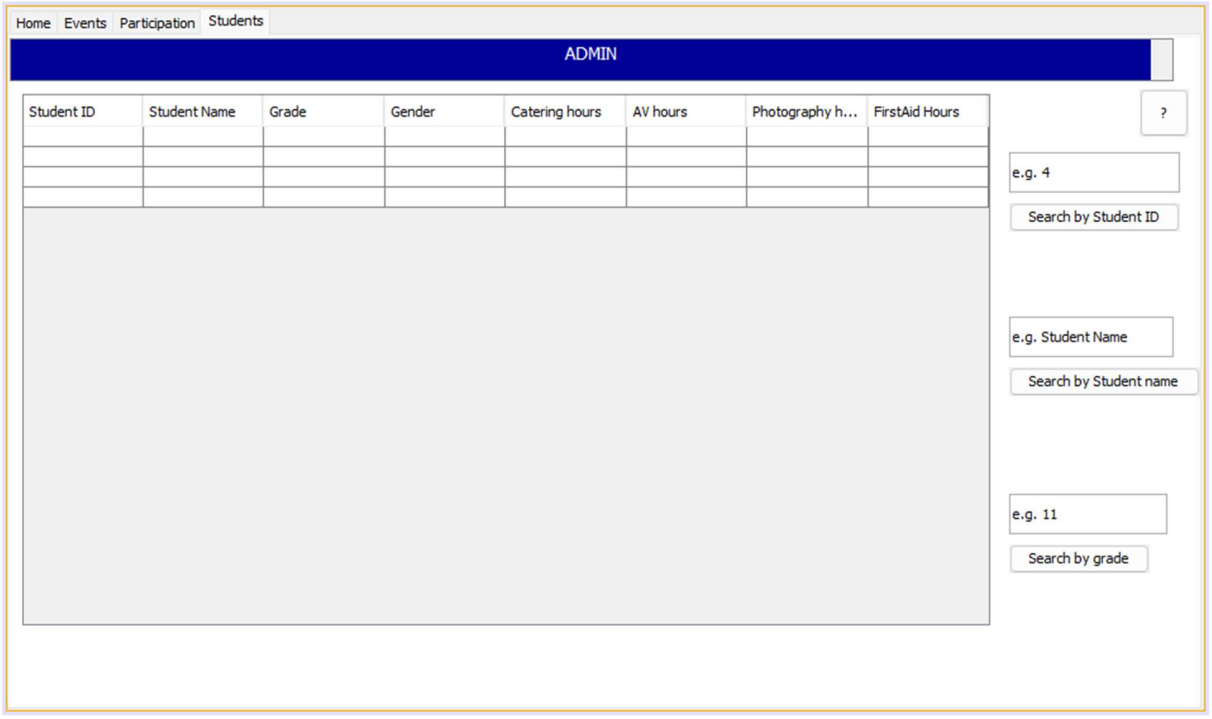
pnlParticipation1 disposed

tbdpStudent disposed

tbdpAdmin disposed

help ← 5

Students page:



1. Search by student ID button pressed:

Connection to the database is made

Default table model set

tblStudents remove all

dtm set row count to 0

tblStudents set to the default table model

```
rs ← "SELECT * FROM tblStudents WHERE StudentID =" + Integer.parseInt(txfSearchStuID.getText())

count ← 0

WHILE rs next

        stu ← rs.getString("StudentID")

        catering ← 0

        photography ← 0

        firstAid ← 0

        av ← 0

        ch ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS CateringHours\n" +
"FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID =
tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Catering =
True\n" +"GROUP BY tblStudents.StudentID;"

        ph ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS PhotographyHours\n" +
"FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID =
tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Photography =
True\n" + "GROUP BY tblStudents.StudentID;"

        avh ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS AVHours\n" + "FROM
tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID = tblStudents.StudentID
AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.AV = True\n" + "GROUP BY
tblStudents.StudentID;"

        fh ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS FirstAidHours\n" +
"FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID =
tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.FirstAid =
True\n" + "GROUP BY tblStudents.StudentID;"

    while ch has next

        IF ((calls a method to check if the admin oversees catering) is true AND stu =
(ch.getString("StudentID")))

            catering ← ch.getInt("CateringHours")
```

```
            END IF
        END WHILE
        WHILE ph has next
            IF ((calls a method to check if the admin oversees photography) is true AND stu =
(ph.getString("StudentID")))
                photography ← ph.getInt("PhotographyHours")
            END IF
        END WHILE
        WHILE avh has next
            IF ((calls a method to check if the admin oversees AV) is true AND stu =
(avh.getString("StudentID")))
                av ← avh.getInt("AVHours")
            END IF
        END WHILE
        WHILE fh has next
            IF ((calls a method to check if the admin first aid) is true AND stu =
(fh.getString("StudentID")))
                firstAid ← fh.getInt("FirstAidHours")
            END IF
        END WHILE
        grade ← rs.getInt("Grade") + "." + rs.getInt("Class")
        name ← rs.getString("FirstName") + " " + rs.getString("Surname")
        cater ← Integer to String (catering)
        aV ← Integer to String (av)
        photo ← Integer to String (photography)
        first ← Integer to String (firstAid)
        IF ((calls a method to check if the admin oversees first aid) is false)
                First ← "NA"
        ELSE IF
        IF ((calls a method to check if the admin oversees AV) is false)
```

```
                aV ← "NA"
            END IF
            IF ((calls a method to check if the admin oversees catering) is false)
               cater ← "NA"
            END IF
            IF ((calls a method to check if the admin oversees first aid) is false)
               photo ← "NA"
            END IF
             Add Row (rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater, aV, photo,first )
             Count ← count +1
END WHILE
        Add Row (Object)
         IF (count = 0)
           Display “Entry not found”
         ELSE IF
```

2. Search by student name button pressed:

Connection to the database is made

Default table model set

tblStudents remove all

dtm set row count to 0

tblStudents set to the default table model

```
      rs ← “SELECT * FROM tblStudents WHERE firstName LIKE '*" + fName + "*' OR surname LIKE '*" + lName + "*'"
       count ← 0
       WHILE rs next
                stu ← rs.getString("StudentID")
                catering ← 0
                photography ← 0
                firstAid ← 0
                av ← 0
```

```
ch ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS CateringHours\n" +
"FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID =
tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Catering =
True\n" +"GROUP BY tblStudents.StudentID;"

ph ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS PhotographyHours\n" +
"FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID =
tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Photography =
True\n" + "GROUP BY tblStudents.StudentID;"

avh ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS AVHours\n" + "FROM
tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID = tblStudents.StudentID
AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.AV = True\n" + "GROUP BY
tblStudents.StudentID;"

fh ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS FirstAidHours\n" +
"FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID =
tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.FirstAid =
True\n" + "GROUP BY tblStudents.StudentID;"

    while ch has next

        IF ((calls a method to check if the admin oversees catering) is true AND stu =
(ch.getString("StudentID")))

            catering ← ch.getInt("CateringHours")

      END IF

    END WHILE

    WHILE ph has next

      IF ((calls a method to check if the admin oversees photography) is true AND stu =
(ph.getString("StudentID")))

        photography ← ph.getInt("PhotographyHours")

      END IF

    END WHILE

    WHILE avh has next

      IF ((calls a method to check if the admin oversees AV) is true AND stu =
(avh.getString("StudentID")))

        av ← avh.getInt("AVHours")

      END IF

    END WHILE
```

```
            WHILE fh has next
               IF ((calls a method to check if the admin first aid) is true AND stu =
(fh.getString("StudentID")))
                  firstAid ← fh.getInt("FirstAidHours")
               END IF
            END WHILE
            grade ← rs.getInt("Grade") + "." + rs.getInt("Class")
            name ← rs.getString("FirstName") + " " + rs.getString("Surname")
            cater ← Integer to String (catering)
            aV ← Integer to String (av)
            photo ← Integer to String (photography)
            first ← Integer to String (firstAid)
            IF ((calls a method to check if the admin oversees first aid) is false)
                  First ← "NA"
            ELSE IF
            IF ((calls a method to check if the admin oversees AV) is false)
                  aV ← "NA"
            END IF
            IF ((calls a method to check if the admin oversees catering) is false)
               cater ← "NA"
            END IF
            IF ((calls a method to check if the admin oversees first aid) is false)
               photo ← "NA"
            END IF
             Add Row (rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater, aV, photo,first )
             Count ← count +1
END WHILE
         Add Row (Object)
          IF (count = 0)
```

Display “Entry not found”

ELSE IF

3. Search by grade button pressed:

Connection to the database is made

Default table model set

tblStudents remove all

dtm set row count to 0

tblStudents set to the default table model

rs ← “SELECT * FROM tblStudents WHERE Grade =" Integer.parseInt(txfSearchStuGrade.getText()

count ← 0

WHILE rs next

stu ← rs.getString("StudentID")

catering ← 0

photography ← 0

firstAid ← 0

av ← 0

ch ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS CateringHours\n" + "FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Catering = True\n" +"GROUP BY tblStudents.StudentID;"

ph ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS PhotographyHours\n" + "FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Photography = True\n" + "GROUP BY tblStudents.StudentID;"

avh ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS AVHours\n" + "FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.AV = True\n" + "GROUP BY tblStudents.StudentID;"

fh ←"SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS FirstAidHours\n" + "FROM tblStudents, tblEvents, tblParticipation\n" + "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.FirstAid = True\n" + "GROUP BY tblStudents.StudentID;"

while ch has next

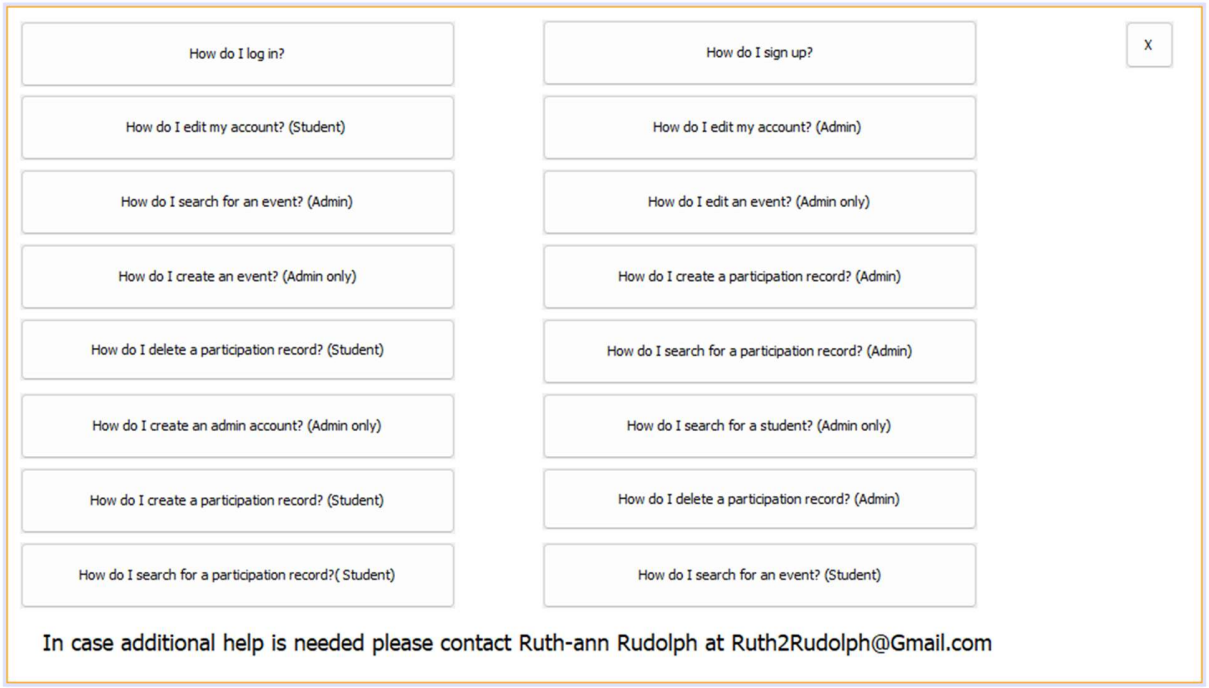
```
            IF ((calls a method to check if the admin oversees catering) is true AND stu =
(ch.getString("StudentID")))
                catering ← ch.getInt("CateringHours")
            END IF
        END WHILE
        WHILE ph has next
            IF ((calls a method to check if the admin oversees photography) is true AND stu =
(ph.getString("StudentID")))
                photography ← ph.getInt("PhotographyHours")
            END IF
        END WHILE
        WHILE avh has next
            IF ((calls a method to check if the admin oversees AV) is true AND stu =
(avh.getString("StudentID")))
                av ← avh.getInt("AVHours")
            END IF
        END WHILE
        WHILE fh has next
            IF ((calls a method to check if the admin first aid) is true AND stu =
(fh.getString("StudentID")))
                firstAid ← fh.getInt("FirstAidHours")
            END IF
        END WHILE
        grade ← rs.getInt("Grade") + "." + rs.getInt("Class")
        name ← rs.getString("FirstName") + " " + rs.getString("Surname")
        cater ← Integer to String (catering)
        aV ← Integer to String (av)
        photo ← Integer to String (photography)
        first ← Integer to String (firstAid)
        IF ((calls a method to check if the admin oversees first aid) is false)
```

```
            First ← "NA"
        ELSE IF
        IF ((calls a method to check if the admin oversees AV) is false)
            aV ← "NA"
        END IF
        IF ((calls a method to check if the admin oversees catering) is false)
           cater ← "NA"
        END IF
        IF ((calls a method to check if the admin oversees first aid) is false)
           photo ← "NA"
        END IF
         Add Row (rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater, aV, photo,first
)
         Count ← count +1
END WHILE
      Add Row (Object)
       IF (count = 0)
            Display “Entry not found”
       ELSE IF
```

4. Help button pressed:

```
pnlHelp set Visible
 pnlStudent disposed
 tbdpStudent disposed
 tbdpAdmin disposed
 help ← 6
```

Help page:



X button pressed:

pnlHelp disposed

switch help

case 1: pnlLogIn set Visible

case 2: pnlSignUp set Visible

case 3: tbdpAdmin set Visible

pnlHomeAdmin set Visible

case 4: tbdpAdmin set Visible

pnlEvents1 set Visible

case 5: tbdpAdmin set Visible

pnlParticipation1 set Visible

case 6: tbdpAdmin set Visible

pnlStudent set Visible

case 7: tbdpStudent set Visible

pnlHome set Visible

case 8: tbdpStudent set Visible

pnlEvents set Visible

case 9: tbdpStudent set Visible

pnlParticipation set Visible

END SWITCH

Help button pressed:

final URI url

url ← new URI("YouTube link to relevant video")

calls a method to open the browser and play the video

Class diagrams

Objects

DBService	
- url: String = "jdbc:ucanaccess://2022PAT.accdb" - conn: Connection = null	Creates a URL to connect to the database Creates a connection to the database
+ DBService() + queryDatabase (sql: String) : ResutlSet + updateQuery (sql: String) : void	Establishes a connection to the database or throws an error Sends and receives SELECT queries between DB and Java Updates the DB using queries in Java

Admin	
- ID : int - adminName: String - adminSurname: String - photography: Boolean - catering : Boolean - firstAid : Boolean - av : Boolean	integer ID of the Admin String first name of the admin String surname of the admin Boolean to check if the admin oversees catering Boolean to check if the admin oversees photography Boolean to check if the admin oversees First Aid Boolean to check if the admin oversees AV
+ Admin (ID: int, adminName: String, adminSurname: String, catering : Boolean, photography: Boolean, av : Boolean, firstAid : Boolean) + getID(): int + getAdminName(): String + getAdminSurname(): String + isPhotography(): Boolean	Parameterized constructor to assign default values to an admin getter method for Admin ID getter method for Admin first name getter method for admin surname getter method for Boolean photography getter method for Boolean Catering

+ isCatering(): Boolean + isFirstAid(): Boolean + toString() : String	getter method for Boolean First Aid getter method for Boolean AV Returns all the details of an admin as a formatted string

Student	
- stuID: Int - stuName: String - stuSurname: String - grade: int - classPoint: int - gender: char - catering: Boolean - photography: Boolean - av: Boolean - firstAid: Boolean	the integer ID of the student String first name of the student String surname of the student Integer grade of the student (8 to 12) Integer class of the student (1 to 5) Character gender of the student (M or F) Boolean indicating if the student does or does not do catering Boolean indicating if the student does or does not do photography Boolean indicating if the student does or does not do AV Boolean indicating if the student does or does not do First Aid
+ Student(stuID: int, stuName: String, grade: int, classPoint: int, gender: char, catering: Boolean, photography: Boolean, av: Boolean, firstAid: Boolean) + getStuID(): int + getStuName(): String + getStuSurname(): String + getGrade(): int + getClassPoint(): int + getGender(): char + isCatering(): Boolean + isPhotography(): Boolean + isAv(): Boolean + isFirstAid() : Boolean + setClassPoint (classPoint: int) + setGender (gender : char) + setCatering(catering: Boolean) + setPhotography (photography: Boolean) + setAv(av: Boolean) + setFirstAid (firstAid: Boolean) + toString(): String	Parameterized Constructor Method to assign values to a student Getter method for the student ID Getter method for the student first name Getter method for the student surname Getter method for the student grade Getter method for the student's class Getter method for the student's gender Getter method for Boolean indicating if the student does or does not do catering Getter method for Boolean indicating if the student does or does not do photography Getter method for Boolean indicating if the student does or does not do AV Getter method for Boolean indicating if the student does or does not do first aid Setter method for the student's gender Setter method for the student's class Setter method for the Boolean indicating if the student does or does not do catering Setter method for the Boolean indicating if the student does or does not do photography Setter method for the Boolean indicating if the student does or does not do AV Setter method for the Boolean indicating if the student does or does not do first aid Returns all the details of an student as a formatted string

Event	
- eventNum: int - eventName: String - numHours: double - date: LocalDate - catering: Boolean - photography: Boolean - firstAid: Boolean - audioVisual: Boolean	Integer event number String name of the event Integer number of hours of the event date of the event Boolean to check if the event is for catering Boolean to check if the event is for photography Boolean to check if the event is for first aid Boolean to check if the event is for AV
+ Event (num: int, n: String, h: double, d: LocalDate, c: Boolean, p: Boolean, f: Boolean, a: Boolean) + getEventNum(): int + getEventName(): String + getNumHours(): double + getDate(): LocalDate + isCatering(): Boolean + isPhotography(): Boolean + isFirstAid(): Boolean + isAudioVisual(): Boolean + toString(): String	parameterized constructor to assign default values to an event getter method for the event number getter method for the event name getter method for hours (number of hours for which the event lasts) getter method for the event date getter method for the Boolean catering getter method for the Boolean photography getter method for the Boolean First Aid getter method for the Boolean Audio Visual Returns all the details of the event as a formatted string

Participation	
- eventID: int - studentID: int	Integer student ID Integer event ID
+ Participation (studentID: int, eventID: int) + getEventID(): int + getStudentID(): int + toString() : String	parameterized constructor to assign default values to a participation record getter method for the event ID of the participation record getter method for the Student ID of the participation record Returns all the details of the participation as a formatted String

Managers

AdminManager	
- size: int = 0 • model: DefaultTableModel = new DefaultTableModel() • d: DBService = new DBService() - AdminArr: Admin [] = new Admin[500]	Stores the size of the array Calls the table model Creates a connection to the database Array to store up to 500 Admin objects

+ add (fn: String, ca: Boolean, p: Boolean, a: Boolean, f: Boolean) + AdminManager () + Shiftleft (pos: int) + insertAdmin (f: admin) +getSize(): int + getPortfolio (c: String, i: int): Boolean + getCurrent(i: int): String + toString(): String	Adds an admin to the array and to the database Parameterized constructor to assign values to an admin Receives a position from which elements will be moved to the left within the array Receives an admin object and creates a query with all relevant details to enter it into the database Returns the size of the array checks if the current Admin oversees the portfolio specified by the string returns a string containing all the details of the Admin with Admin ID equal to I To String method returns all Admins in the array

StudentManager	
- Size: int = 0 • Model: DefaultTableModel = new DefaultTableModel() • d: DBService = new DBService() - StudentArr: Student [] = new Student [1000]	Stores the size of the array Calls the table model Creates a connection to the database Array to store up to 1000 Student objects
+ add(fn: String, sn: String, g: int, c: int, gen: char, ca: Boolean, p: Boolean, a:Boolean, f: Boolean) + StudentManager() + shiftLeft(pos: int) + getLast(): int + insertStudent(f: Student) + getCurrent(i: int): String +getSize(): int + getPortfolio(c: String, i: int): Boolean + toString(): String	adds a new Student object to the database and the array Assigns values to each student object and adds them to the array Receives a position from which elements will be moved to the left within the array Returns the ID of the last student in the array Receives a student object and creates a query with all relevant details to enter it into the database Returns a string containing all the details of the student with student ID equal to i Returns the size of the array checks if the current Student (position in database at i) participates in the portfolio specified by the string returns all Students in the array

EventManager	
- Size: int = 0 • Model: DefaultTableModel = new DefaultTableModel() • d: DBService = new DBService()	Stores the size of the array Calls the table model Creates a connection to the database

- EventArr: Event[] = new Event[1000]	Array to store up to 1000 Event objects
+ add (n: String, h: double, d: LocalDate, c: Boolean, p: Boolean, f: Boolean, a: Boolean) + EventManager() + shiftLeft(pos: int) + insertEvent(f: Event) +getSize(): int + toString(): String	Receives all information for a new event except the auto generated Event number Constructor Method to assign Events to an array from the database Receives a position from which elements will be moved to the left within the array Receives an event object and creates a query with all relevant details to enter it into the database Returns the size of the array returns all events in the array

ParticipationManager	
- Size: int = 0 • Model: DefaultTableModel = new DefaultTableModel() • d: DBService = new DBService() - PartArr: Participation [] = new Participation [2000]	Stores the size of the array Calls the table model Creates a connection to the database Array to store up to 2000 participation objects
+ add (StudentID: int, eventID: int) + ParticipationManager() + shiftLeft(pos: int) + insertParticipation(s: int, e: int) +DeleteEntry(event: int, student: int): String + getSize(): int + toString(): String	adds a new participation object to the array and database Creates an array of participation objects with all their relevant details from the database Receives a position from which elements will be moved to the left within the array Receives an object and creates a query with all relevant details to enter it into the database Receives a row number and deletes the corresponding entry in the database and the object array Returns the size of the array returns all Participation records in the array

Persistent storage design

1. tblStudent

Field	Data Type	Description	Example
StudentID	Integer autoincrement	Unique identifier for the student	1
FirstName	Short Text	First name of the student	Ruth-ann
Surname	Short Text	Surname of the student	Rudolph

Grade	Integer	Grade of the student (8-12)	12
Class	Integer	Class number of the student (1-5)	1
Gender	Short Text	Gender of the student (M/F)	F
Catering	Boolean	A true of false indication of whether the person participates I the portfolio	True
Photography	Boolean	A true of false indication of whether the person participates I the portfolio	False
AV	Boolean	A true of false indication of whether the person participates I the portfolio	False
FirstAid	Boolean	A true of false indication of whether the person participates I the portfolio	True

2. tblAdmin

Field	Data Type	Description	Example
AdminID	Integer autoincrement	Unique identifier for the admin member	5
AdminName	Short Text	First name of the admin member	Terri
AdminSurname	Short Text	Surname of the admin member	Dennis
Catering	Boolean	A true of false indication of whether the admin oversees the portfolio	False
Photography	Boolean	A true of false indication of whether the admin oversees the portfolio	False

AV	Boolean	A true of false indication of whether the admin oversees the portfolio	True
FirstAid	Boolean	A true of false indication of whether the admin oversees the portfolio	True

3. tblEvent

Field	Data Type	Description	Example
EventID	Integer autoincrement	Unique identifier for event	10
EventName	Short Text	Name of event	Open day
EventDate	Date	Date of the event	24/03/2022
Portfolio	Short Text	Name of portfolio which relates to the event	Catering
Hours	Integer	Number of hours the event counts for	6

4. tblParticipation

Field	Data Type	Description	Example
EventID	Integer	Unique identifier for the event	10
StudentID	Integer	Unique identifier for the student	1

5. UsernamesAndPasswords.txt

Format: ID#username#password#Title

Example data:

1#cha11112.1@curro.co.za#hillcrest1234#Student

1#Origanal#Ruth#Admin

2#Kieran123#Ruthie#Student

3#SkyePie#hillcrest1234#Student

4#cha11cun004.1@curro.co.za#hillcrest1234#Student

All data to be stored locally.

Explanation of Secondary Storage Design

Most of the information is stored in the database because it is complex with intricate relationships and is best stored in a normalized manner so that it can be easily edited and read, through SQL queries. Storage of the information in the database allows for links through primary and foreign keys, which also help to simplify complex SQL queries.

The usernames and passwords are stored in a text file because the data is small in value and simple, it will also not be changed other than the addition of new accounts.

Explanation of Primary Data Structure related to Secondary Storage

Primary memory

Student Class

- The student class is needed to store the information of the students
- Creates a student object and assigns 10 fields to it. (ID, name, surname, grade, class, catering, photography, av and first aid)
- This class contains getter methods for all fields and setter methods for all fields except ID because ID is an auto generated field in the database
- The class is used in the studentManager class

Admin Class

- The Admin class is needed to store the information of the Admins
- Creates an admin object and assigns 7 fields to it. (ID, name, surname, catering, photography, av and first aid)
- This class contains getter methods for all fields
- The class is used in the AdminManager class

Event Class

- The events class is needed to store the information of the events
- Creates an event object and assigns 8 fields to it. (Number, name, hours, date, catering, photography, av and first aid)
- This class contains getter methods for all fields
- The class is used in the EventManager class

Participation Class

- The participation class is needed to store the information of the participation records
- Creates a student object and assigns 2 fields to it. (EventID and StudentID)
- This class contains getter methods for all fields
- The class is used in the ParticipationManager class

DB Class

- The DB class allows the user to access the database using SQl statements
- This class is accessed by frmMain, AdminManager class, the EventManager class, the ParticipationManager class and the StudentManager class

StudentManager class

- Creates an array of student objects
- Populates the array with the information in the database
- This class contains methods that extract various fields from the array or specific objects within the array, where necessary, as well as allowing for addition of fields to the array and the database

AdminManager class

- Creates an array of Admin objects
- Populates the array with the information in the database
- This class contains methods that extract various fields from the array or specific objects within the array, where necessary, as well as allowing for addition of fields to the array and the database

EventManager class

- Creates an array of event objects
- Populates the array with the information in the database
- This class contains methods that extract various fields from the array or specific objects within the array, where necessary, as well as allowing for addition of fields to the array and the database

ParticipationManager

- Creates an array of participation objects
- Populates the array with the information in the database
- This class contains methods that extract various fields from the array or specific objects within the array, where necessary, as well as allowing for addition of fields to the array and the database

Secondary Memory

Students table

- This table stores all the information of all the students who have accounts, uniquely identified by the student ID (ID, name, surname, grade, class, and portfolio selection)
- This is accessed from all parts of the program on the student's side of the program, apart from the participation page. The home page to display the users' details, the edit account page where the user can edit their details, the events page where the user can only view events related to their portfolio selection.
- It is needed so that each user's information can be easily accessed and edited, in a normalized format

Admins table

- This table stores all the information of the admins (ID, name, surname, and portfolio selection), uniquely identified by the admin ID which links to the text file
- This is accessed from almost all parts of the program on the admin side of the program. The home page to display the users' details, the edit account page where the user can edit their details, the events page where the user can only view events related to their portfolio selection and the students page where the admin can view all the students who participate in the portfolios they oversee.
- It is needed so that each user's information can be easily accessed and edited, in a normalized format

Events table

- This table stores all the information of each event, uniquely identified by the event ID.
- This information is accessed from the event page of the admin and the student, as well as the participation page where the details of the event are displayed to make the program more user friendly.
- It is needed so that each event's information can be easily accessed and edited, in a normalized format.

Participation table

- This table stores the Student ID and event ID, so that a record of which event each student participates in can be easily stored and manipulated.
- This information is only accessed from the student home page and the participation page of both the student and the admin.
- It is needed so that a close record of participation can be kept and edited easily.

UsernamesAndPasswords.txt

- Stores the ID, username, password, and title (Student or Admin) for each user
- The ID links the user's information on the text file and the database, the username is used to identify the user when they log in, the password is used to verify the user and the title is to identify for what the user has permissions.
- The text file is accessed when the user logs in, signs up or when an admin creates a new admin account.